

Universitatea „Sapientia” din Cluj-Napoca
FACULTATEA DE ȘTIINȚE TEHNICE ȘI UMANISTE,
TÎRGU MUREȘ
SPECIALIZAREA CALCULATOARE

Recunoașterea automată a instrumentelor muzicale pe baza înregistrărilor audio

PROIECT DE DIPLOMĂ

Coordonator științific:
Conf. univ. dr. Iclănzan David
Andrei

Absolvent:
Nagy Csaba

LUCRARE DE DIPLOMĂ

Coordonator științific:
Conf. univ. dr. Iclănzan David Andrei

Candidat: **Nagy Csaba**
Anul absolvirii: **2026**

a) Tema lucrării de licență:

Recunoașterea automată a instrumentelor muzicale pe baza înregistrărilor audio

b) Problemele principale tratate:

- Studiul teoretizat al procesării semnalelor audio și al algoritmilor de învățare profundă (Deep Learning).
- Construirea unui set de date hibrid prin fuziunea a șase baze de date publice (IRMAS, NSynth, Medley-solos-DB, TinySOL, Good-sounds, ESC-50) și înregistrări proprii realizate cu microfon.
- Extragerea și analiza caracteristicilor spectrale: Log-Mel spectrograme, Short-Time Fourier Transform (STFT) și Mel-Frequency Cepstral Coefficients (MFCC).
- Proiectarea, optimizarea și antrenarea unei arhitecturi de rețea neuronală convoluțională bidimensională (2D CNN) în mediul Google Colab.
- Dezvoltarea unui modul de augmentare a datelor (zgomot de fond, reverberație) pentru îmbunătățirea robusteții modelului în condiții acustice reale.
- Validarea sistemului prin implementarea unui modul de recunoaștere în timp real a sunetelor captate la microfon.

c) Desene obligatorii:

d) Softuri obligatorii:

- Prelucrarea setului de date audio în Python 3.
- Proiectarea și antrenarea rețelei neuronale 2D CNN cu TensorFlow și Keras în Google Colab.
- Extragerea caracteristicilor spectrale ale sunetelor utilizând biblioteca Librosa.
- Dezvoltarea modulului de recunoaștere în timp real a sunetelor cu SoundDevice.

e) Bibliografia recomandată:

f) Termene obligatorii de consultații: săptămânal

g) Locul și durata practicii: Universitatea Sapientia,
Facultatea de Științe Tehnice și Umaniste din Târgu Mureș

Primit tema la data de:

Termen de predare:

Semnătura Director Departament

Semnătura coordonatorului

**Semnătura responsabilului
programului de studiu**

Semnătura candidatului



Declarație

Subsemnatul **Nagy Csaba**, absolvent al specializării **Calculatoare**, promoția **2026** cunoscând prevederile Legii Educației Naționale 1/2011 și a Codului de etică și deontologie profesională a Universității Sapientia cu privire la furt intelectual declar pe propria răspundere că prezenta lucrare de licență/proiect de diplomă/disertație se bazează pe activitatea personală, cercetarea/proiectarea este efectuată de mine, informațiile și datele preluate din literatura de specialitate sunt citate în mod corespunzător.

Târgu Mureș,

Data:

Semnătura:



Recunoașterea automată a instrumentelor muzicale pe baza înregistrărilor audio

Extras

Prezenta lucrare de diplomă abordează problema recunoașterii automate a instrumentelor muzicale pe baza înregistrărilor audio, utilizând metode moderne de învățare automată. Sistemul dezvoltat este capabil să clasifice în timp real șapte categorii de instrumente (chitară, pian, voce, instrumente cu coarde, instrumente cu ancie, instrumente de alamă și zgomot ambiental) pornind de la semnale audio monofonice.

Abordarea propusă se bazează pe extragerea a trei tipuri de caracteristici spectrale (Log-Mel spectrograme, STFT și MFCC) și pe antrenarea unor rețele neuronale convoluționale bidimensionale (2D CNN) în mediul Google Colab. Setul de date hibrid a fost creat prin combinarea unor eșantioane curate descărcate de pe internet, a unor re-înregistrări proprii prin microfon și a zgomotului ambiental din baza de date ESC-50, iar un modul de augmentare a datelor simulează condițiile reale de captare prin adăugarea de reverberație, zgomot ambiental și distorsiuni de microfon.

Rezultatele experimentale arată o acuratețe competitivă a modelului pe datele de test, iar sistemul a fost validat prin recunoașterea în timp real a sunetelor captate cu microfonul laptopului. Lucrarea contribuie la demonstrarea fezabilității unui clasificator robust de instrumente muzicale, antrenat cu resurse computaționale accesibile.

Cuvinte cheie: recunoașterea instrumentelor muzicale, rețele neuronale convoluționale, Mel-spectrogramă, STFT, MFCC, clasificare audio, formă de undă, imagine spectrală

Sapientia Erdélyi Magyar Tudományegyetem

MAROSVÁSÁRHELYI KAR

SZÁMÍTÁSTECHNIKA SZAK

Automatikus hangszerfelismerés zenefelvételek alapján

Diplomadolgozat

Témavezető:

**Dr. Iclănzan David Andrei,
egyetemi docens**

Végzős hallgató:

Nagy Csaba

2026

Automatikus hangszerfelismerés zenefelvételek alapján

Kivonat

Jelen diplomadolgozat az automatikus hangszerfelismerés problémáját vizsgálja zenei felvételek alapján, modern gépi tanulási módszerek alkalmazásával. A kidolgozott rendszer valós időben képes osztályozni hét hangszerkategóriát (gitár, zongora, ének, vonós, nádas fúvós, rézfúvós és környezeti zaj) monofón audiojel alapján.

A megvalósított megoldás három spektrális jellemző típust (Log-Mel spektrogram, STFT és MFCC) használ, amelyek kétdimenziós konvolúciós neurális hálózatba (2D CNN) kerülnek betáplálásra. A modell tanítása Google Colab környezetben történt. Az adathalmazt saját tiszta internetes minták és mikrofonnal/telefonnal újrafelvételezett minták, valamint az ESC-50 környezeti zajadatbázis egyesítésével hoztam létre. Az adataugmentációs modul a valós felvételi körülményeket szimulálja visszhang, környezeti zaj és mikrofonkarakterisztika hozzáadásával.

A kísérleti eredmények versenyképes osztályozási pontosságot mutatnak a tesztadatokon, és a rendszert valós idejű mikrofonos felismeréssel is validáltam. A dolgozat hozzájárul annak demonstrálásához, hogy elérhető számítási erőforrásokkal is megvalósítható egy robusztus hangszerosztályozó rendszer.

Kulcsszavak: hangszerfelismerés, konvolúciós neurális hálózat, Mel-spektrogram, STFT, MFCC, audioosztályozás, hullámforma, spektrális kép

Automatic Musical Instrument Recognition Based on Audio Recordings

Abstract

This thesis addresses the problem of automatic musical instrument recognition from audio recordings using modern machine learning methods. The developed system is capable of classifying seven instrument categories in real time (guitar, piano, vocals, strings, reeds, brass, and ambient noise) from monophonic audio signals.

The proposed approach relies on the extraction of three types of spectral features (Log-Mel spectrograms, STFT, and MFCC), which are fed into two-dimensional convolutional neural networks (2D CNN) trained in the Google Colab environment. A hybrid dataset was assembled by combining clean internet-sourced samples, self-recorded microphone playback samples, and ambient noise from the ESC-50 database. A data augmentation module simulates real-world recording conditions by adding reverberation, ambient noise, and microphone characteristics.

Experimental results demonstrate competitive classification accuracy on the test set, and the system has been validated through real-time recognition of sounds captured by a laptop microphone. This work contributes to demonstrating the feasibility of building a robust musical instrument classifier using accessible computational resources.

Keywords: musical instrument recognition, convolutional neural network, Mel-spectrogram, STFT, MFCC, audio classification, waveform, spectral image

Előszó

Amióta az eszemet tudom, a reáltudományok és a művészetek – különösen a zene – világa meghatározó szerepet játszik az életemben. Időről időre az egyik kerül előtérbe, majd a másik veszi át a szerepet, de sosem maradt ki teljesen egyik sem az érdeklődésemből. Ez az oka, hogy sosem tudtam csak egy oldal mellett elköteleződni véglegesen és minden fókuszomat kizárólag ráhelyezni, hogy minél jobb legyek benne. Talán nincs is rá szükség. A reáltudományok segítenek leírni a körülöttem lévő fizikai valóságot és lépést tarthatok vele a technológia exponenciális előretörésével. Ennek ellenére mégis hiányérzetem támad, ha a saját belső világom szeretném megfigyelni. A művészetek körülírják és teljesebbé teszik az ember lelkivilágát, így teremtve meg számomra az egyensúlyt.

Ez az egyensúlykeresés inspirálta jelen kutatásomat és diplomadolgozatomat is, amely a zene és a számítástechnika határterületén valósul meg.

Nagy Csaba

TARTALOMJEGYZÉK

Előszó	iii
Tartalomjegyzék	iv
Ábrák jegyzéke	vii
Táblázatok jegyzéke	ix
1. Bevezető	1
2. Elméleti megalapozás és szakirodalmi tanulmány	3
2.1. Szakirodalmi tanulmány	3
2.2. Elméleti alapok: Audio jelfeldolgozás	3
2.2.1. A hang digitalizálása (ADC)	3
2.2.2. A jelek előkészítése: Keretezés és Ablakozás	4
2.2.3. Időtartománybeli jellemzők	5
2.2.4. Hangszer családok akusztikai indoklása és hullámformái	5
2.2.5. Frekvenciatartománybeli jellemzők	7
2.2.6. Percepció jellemzők és a Mel-spektrogram	9
2.3. Ismert hasonló alkalmazások	11
2.4. Felhasznált technológiák és eszköztár	11
3. A rendszer specifikációi és architektúrája	12
3.1. Célkitűzések és követelményelemzés	12
3.1.1. Funkcionális követelmények	12
3.1.2. Nem funkcionális követelmények	13
3.2. Használati esetek (Use Case modell)	13
3.3. Rendszerarchitektúra és adatfolyam	13
3.4. Alkalmazott technológiák és fejlesztői környezet	16
3.4.1. Programozási nyelv és alapkörnyezet	16
3.4.2. Akusztikai jelfeldolgozás: Librosa	16
3.4.3. Mélytanulási keretrendszer: TensorFlow és Keras	16
4. Részletes tervezés	18
4.1. Az adathalmaz tervezése	18
4.1.1. Saját készítésű organikus adathalmaz	18
4.1.2. Mikrofonos újrafelvétel a valós idejű kiértékelés felkészítésének érdekében	21
4.1.3. Végző adathalmaz összeállítása	22
4.2. Jellemzőkinyerés tervezése	23

4.2.1.	Log-Mel spektrogram	23
4.2.2.	STFT spektrogram	24
4.2.3.	MFCC (Mel-Frequency Cepstral Coefficients)	24
4.2.4.	Nyers hanghullám (Raw waveform)	24
4.2.5.	A megfelelő jellemzőkinyerési eljárás kiválasztása	24
4.2.6.	Normalizálás	25
4.2.7.	Jellemzőkinyerés kimeneti állományai	26
4.3.	Adataugmentáció tervezése	26
4.3.1.	Szobai reverb szimuláció	26
4.3.2.	Frekvenciaválasz-torzítás (Pre-emphasis)	27
4.3.3.	Környezeti zajkeverés (Noise mixing)	27
4.3.4.	Szoftklip torzítás (Soft clipping)	27
4.3.5.	Végső normalizálás	27
4.4.	Modell architektúrák tervezése	29
4.4.1.	A hálózati architektúra részletei	29
4.4.2.	YAMNet Transfer Learning architektúra	30
4.4.3.	Tanítási konfiguráció	31
4.4.3.1.	A tanítás során generált kimenetek	32
4.5.	A valós idejű felismerő modul tervezése	33
4.5.1.	Audio stream és aszinkron pufferek	33
4.5.2.	Jellemzőkinyerés és normalizálás	34
4.5.3.	Zaj-alapú küszöbölés és Predikciós stabilizáció	34
4.6.	A feldolgozási folyamat összefoglaló áttekintése	35
5.	Üzembe helyezés és kísérleti eredmények	37
5.1.	Üzembe helyezési lépések	37
5.2.	Felmerült problémák és megoldások	38
5.3.	Kísérleti eredmények és mérések	39
5.3.1.	A 2D CNN és a YAMNet modellek összehasonlítása	40
6.	A rendszer felhasználása	41
6.1.	Az alkalmazás indítása és konfigurációja	41
6.2.	A parancssori felhasználói felület felépítése	41
6.2.1.	ANSI színkódolás és jelölések	42
6.3.	Működési forgatókönyvek és tesztesetek	42
6.3.1.	1. teszteset: Csendes szobai környezet (alapállapot)	43
6.3.2.	2. teszteset: Akusztikus gitárjáték (monofón dallam)	43
6.3.3.	3. teszteset: Emberi énekhang (vokál)	43
6.3.4.	4. teszteset: Szaxofon játék (átmeneti bizonytalanság)	43
6.3.5.	Az alkalmazás leállítása	44
7.	Következtetések	45
7.1.	Megvalósítások	45
7.2.	Hasonló rendszerekkel való összehasonlítás	45

7.3. Továbbfejlesztési lehetőségek	46
8. Irodalomjegyzék	47
9. Függelék	48
9.1. A projekt könyvtárszerkezete	48
9.2. A feldolgozási csővezeték szkriptjei	49
9.3. Fejlesztői és futtatási környezet	50

ÁBRÁK JEGYZÉKE

2.2.1.A keretezés (framing) és az ablakozás (windowing) folyamata: a jelet átfedő keretekre bontjuk, majd Hann-ablakot alkalmazunk a spektrális szivárgás elkerülése érdekében	4
2.2.2.A rendszerben használt főbb osztályok mikroszkopikus (5 ms-os) periodikus hullámformáinak összehasonlítása. A vizuális konzisztencia érdekében minden hangszermintát azonos, 440 Hz-es (A4) alapfrekvenciára transzponáltunk. Jól megfigyelhető, hogy a periódusidők kiegyenlítése után az egy osztályba tartozó hangszerek hullámformájának alapszerkezete mekkora hasonlóságot mutat.	8
2.2.3.Egy zongoraakkord STFT-alapú spektrogramja: a vízszintes tengely az időt, a függőleges tengely a frekvenciát, a szín intenzitása pedig az energiát (dB) jelöli	9
2.2.4.Egy hangfelvétel lineáris frekvenciaskálájú (felső) és Mel-skálájú (alsó) spektrogramjának összehasonlítása. A Mel-skála az alacsony frekvenciákon finomabb felbontást biztosít, így az ember számára releváns hangmagasság-információt hűebben tükrözi.	10
3.2.1.A hangszerfelismerő rendszer főbb használati esetei (UML Use Case diagram)	14
3.3.1.A rendszerarchitektúra és a folyamatos adatfolyam (UML Komponensdiagram)	15
4.1.1.A saját gyűjtésű organikus adathalmaz eloszlása a szeparált szeletelés után .	20
4.1.2.A mikrofonos újrafelvételi és szinkronizált szeletelési folyamat lépései . . .	22
4.1.3.A saját tiszta és az újrafelvételezett zajos minták megoszlása az adathalmazban	22
4.2.1.A három vizsgált jellemzőtípus vizualizációja ugyanazon gitárminta esetén: (a) STFT spektrogram (513×32), (b) normalizált Log-Mel spektrogram (64×32), (c) normalizált MFCC együtthatók (40×32)	25
4.3.1.A MacBook/mikrofonos felvételi körülményeket szimuláló adataugmentációs folyamatlanc	27
4.3.2.A hat hangszerosztály (gitár, zongora, vokál, hegedű, klarinét, kürt) egy-egy reprezentatív mintájának normalizált Log-Mel spektrogramja az eredeti (tiszta) és az adataugmentált (MacBook/mikrofon szimulációs láncsal torzított) állapotban	28
4.4.1.A tervezett 2D CNN hálózati architektúra felépítése és adatfolyama	30
4.5.1.A valós idejű felismerési folyamat lépései az élő hangbeviteltől a kijelzésig .	35
4.6.1.A hangszerfelismerő rendszer végső adatfeldolgozási és tanítási folyamatának lépései	36

5.3.1.A Log-Mel spektrogram alapú 2D CNN modell konfúziós mátrixának hő térképe	39
6.1.1.Az alkalmazás indításakor megjelenő konfigurációs és betöltési paraméterek	41
6.3.1.A rendszer kimenete csendes szobai környezet (zaj) esetén	43
6.3.2.A rendszer kimenete akusztikus gitárjáték érzékelésekor	43
6.3.3.A rendszer kimenete emberi énekhang (vokál) érzékelésekor	43
6.3.4.A rendszer kimenete szaxofon játék (reed) érzékelésekor	43

TÁBLÁZATOK JEGYZÉKE

2.2.1. Az analóg-digitális konverzió (ADC) legfontosabb paraméterei	4
2.2.2. A rendszerben használt hangszercsaládok, a hangkeltés mechanizmusa és a hullámforma jellemzői	6
2.2.3. Mel-spektrogram és MFCC összehasonlítása gépi tanulási szempontból . . .	11
4.1.1. A vizsgált publikus adathalmazok összehasonlító elemzése	19
4.2.1. A három vizsgált jellemzőkinyerési módszer összehasonlító elemzése . . .	25
4.4.1. A tervezett 2D CNN architektúra rétegei (Log-Mel bemenet: $64 \times 32 \times 1$) .	29
4.4.2. A modell tanításához használt konfigurációs paraméterek és beállítások . .	32
5.3.1. A két fő modellarchitektúra teljesítményének és paramétereinek összehasonlítása	40
6.2.1. A parancssori felületen alkalmazott ANSI színkódok és rövidítések kategóriánként	42
9.1.1. A projekt legfelső szintű könyvtárszerkezete	48
9.2.1. A feldolgozási csővezeték szkriptjei és feladataik	49
9.3.1. A fejlesztői környezet főbb összetevői	50

1. BEVEZETŐ

Kodály Zoltán szerint a zene hangzó matematika. Hasonló analógiával élve, a számítástechnika pedig a digitális matematika egy formája, ami számomra azt mutatja meg, hogy valójában mindkét oldal közös tőről fakad.

Kutatásom és ezen dolgozat elkészítése pontosan ebben a metszéspontban, a zene és a számítástechnika határterületén valósul meg. Ezt a közös határterületet a tudományos világban a zenei információ-visszanyerés (Music Information Retrieval – MIR) interdiszciplináris területe képviseli, amely a digitális jelfeldolgozás és a gépi tanulás eszköztárát ötvözi, hogy képessé tegye a számítógépes rendszereket az audiojelek automatikus értelmezésére. Ezen belül az automatikus hangszer- és hangszercsalád-felismerés az egyik legizgalmasabb kutatási irány, amely kulcsfontosságú a digitális zenei adatbázisok intelligens kereshetőségéhez, a valós idejű zeneelemzéshez, valamint az automatikus kottázáshoz (transzkripcióhoz).

Hétköznapi szinten ennek a technológiának az egyik leglátványosabb alkalmazása a modern zenei streaming szolgáltatók (mint például a Spotify vagy az Apple Music) ajánló-rendszereiben érhető tetten. Ezek a platformok kifinomult algoritmusok segítségével elemzik és szabják személyre az általunk preferált zenei mintázatokat, hangszereket és dalokat, megteremtve ezzel a zenehallgatás új, intelligens korszakát. Ennek a személyre szabási folyamatnak a legalapvetőbb lépcsőfokát az olyan alacsony szintű zenei attribútumok és akusztikai jellemzők automatikus felismerése alkotja, mint amilyen a felvételeken megszólaló hangszerek azonosítása is.

A zenében a hangszerek változatos környezetben szólhatnak meg. Háttér szempontjából beszélhetünk „laboratóriumi” stúdiófelvételekről vagy valós idejű, környezeti háttérzajjal terhelt jelekről. A stúdióminőséghez jó analógia a légüres tér, ahol ideális körülmények között csak igencsak minimális zaj található meg, míg normális esetben szinte mindig zaj kíséri a zenét – ha más nem, legalább olyan természeti hangok, mint a szél, a madárcsicsergés vagy a lépések zaja, illetve magának a hangszernek a fizikai megszólaltatása, ami a hangfeldolgozás esetében a második legnagyobb probléma. A hangszerek megszólalhatnak egyedül, de akár többen is egyszerre: több hangszer együttes megszólalása egymásra adódik, úgymond összeolvad, aminek a szétválasztása és megkülönböztetése – mint a hangszerfelismerés „első számú károkozója” – különösen orkesztrák esetén szinte lehetetlen.

A probléma orvoslására az utóbbi időkben a deep learning modellek, különösen a 2D CNN hozott nagy könnyebbiséget, amely a hang szeleteiből hoz létre egy-egy spektrális képet, mintegy a hangok ujjlenyomataként. A hangokat képpé alakítva, a vizuális mintázatok és karakterisztikák alapján történő elemzés lehetővé teszi a hangszerek sajátos színezetének felismerését, ám a szélsőségesen komplex zenekaroknál ez a módszer sem jelent teljes áttörést.

A kutatáshoz használt adathalmazt saját magam állítottam össze az interneten fellelhető zenefelvételek egészen apró szeleteiből, miután alaposan áttekintettem a korábban létező adatbázisokat. Azok a meglévő minták ugyanis nem követték pontosan az én tervezési szándékaimat, és nem adtak választ azokra a specifikus problémákra, amelyeket vizsgálni

szerettem volna. Hosszú távon sokkal célravezetőbb egy teljesen saját, organikus adatbázis felépítése, amely tökéletesen illeszkedik a rendszer egyedi igényeihez.

Az elkészült neurális hálózatra és a saját adathalmazra építve a kutatás végcélja egy olyan alkalmazás kifejlesztése, amely közvetlen, gyakorlati segítséget nyújthat a felhasználóknak. Egy ilyen rendszer továbbfejlesztett változata különösen hasznos lehet azok számára, akik ritkán fordultak meg valós, élő zenei környezetben, és fejletlen a hangszerfelismerési képességük, de gyakran találják magukat olyan helyzetben, amikor egy számukra tetsző, videóklip nélküli dallamban megszólaló hangszer nevét szeretnék azonosítani.

A dolgozat további fejezetei lépcsőről lépésre követik a szoftverfejlesztés lépéseit. A zenei és gépi tanulási elmélet megalapozása után részletesen bemutatásra kerül a rendszer specifikációja és architektúrája, amelyet a 2D CNN hálózat és a valós idejű alkalmazás implementációja, valamint a gyakorlati tesztelés és kiértékelés követ.

2. ELMÉLETI MEGALAPOZÁS ÉS SZAKIRODALMI TANULMÁNY

Ez a fejezet tartalmazza az elvégzett dokumentálódás és kutatómunka eredményeit, különös tekintettel az audio jelfeldolgozás és a gépi tanulás kapcsolatának elméleti hátterére, valamint a felhasznált technológiák ismertetésére. A bemutatott akusztikai jelfeldolgozási algoritmusok és azok neurális hálózatokra történő leképezése jelentős részben Valerio Velardo *The Sound of AI* című átfogó nyílt oktatási anyagán [1] alapulnak.

2.1. Szakirodalmi tanulmány

2.2. Elméleti alapok: Audio jelfeldolgozás

A hangok gépi tanulási modellekkel történő feldolgozásához elengedhetetlen a fizikai hanghullámok megfelelő digitális reprezentációinak előállítása és a releváns jellemzők kinyerése. Az alábbiakban a hang digitális feldolgozásának főbb lépéseit mutatom be.

2.2.1. A hang digitalizálása (ADC)

A fizikai világban a hang egy analóg, mechanikai hullám, amely időben és amplitúdóban is folytonos. A gépi tanulási modellek feldolgozásához ezt az Analóg-Digitális Konverzió (ADC) folyamatával diszkrét digitális adattá kell alakítani. Ennek a folyamatnak két fő fázisa van:

- **Mintavételezés (Sampling):** Az időtengely diszkrétizálása, melynek során a folyamatos jelből egyenlő időközönként pillanatfelvételeket rögzítünk. Ez a mintavételezési frekvencia (Sample Rate). A Nyquist-tétel értelmében a rögzíthető legmagasabb frekvencia a mintavételezési frekvencia fele, azaz a Nyquist-frekvencia (például hagyományos CD-k 44 100 Hz-es mintavételezése esetén 22 050 Hz). Ha ennél magasabb frekvenciakomponensek maradnának a jelben, az súlyos akusztikus torzítást, úgynevezett *aliasing*-et (szellemképződést) okozna. Ennek elkerülése érdekében a hardveres ADC eszközök aluláteresztő (anti-aliasing) szűrővel vágják le a Nyquist-frekvencia feletti tartományt.
- **Kvantálás (Quantization):** A mintavett analóg amplitúdóértékek legközelebbi diszkrét szintre történő kerekítése. A kvantálás felbontását a bitmélység (Bit Depth) adja meg, amely meghatározza a rögzíthető leghalkabb és leghangosabb hang közötti dinamikatartományt. A szabványos 16 bites felbontás (2^{16} , azaz 65 536 lehetséges érték) kiváló, közel 96 dB-es Jel-Kvantálási Zaj Viszonyt (SQNR) és dinamikatartományt biztosít.

A 2.2.1. táblázat összefoglalja a digitalizálás legfontosabb paramétereit és azok hatásait.

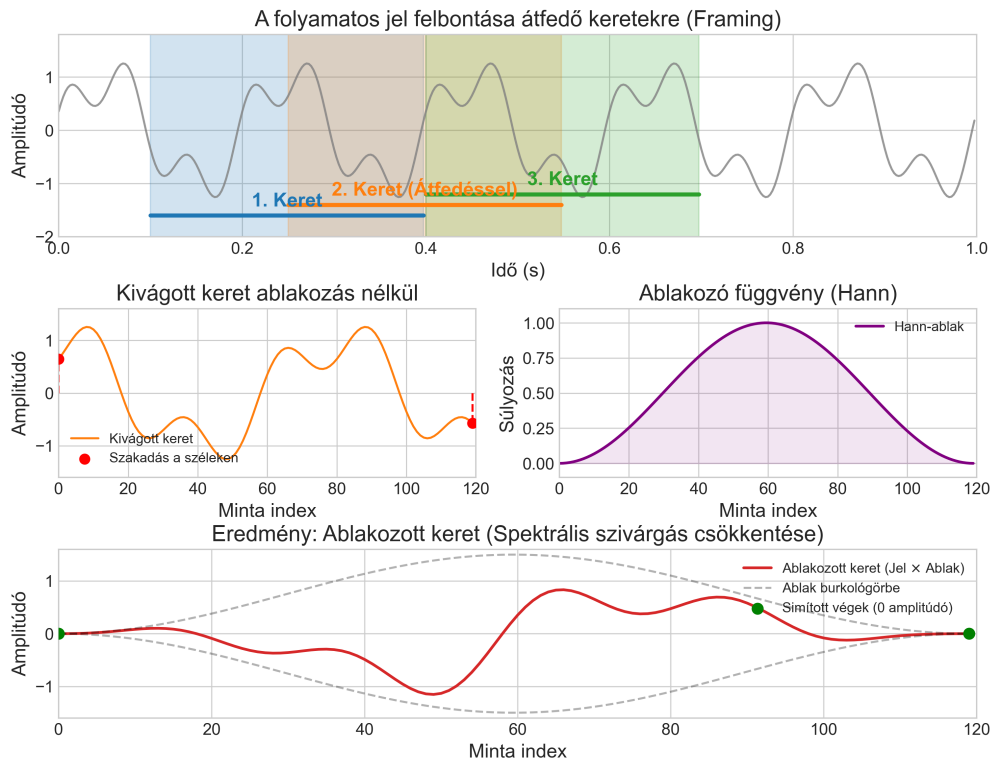
2.2.1. táblázat: Az analóg-digitális konverzió (ADC) legfontosabb paraméterei

Paraméter	Tipikus érték (CD)	Hatása
Mintavételezési freki.	44 100 Hz	Max. rögzíthető freki.: 22 050 Hz
Bitmélység	16 bit	Dinamikatartomány: ~96 dB
Csatornaszám	2 (sztereó)	Térbeli információ megőrzése
Nyers adatméret (1 perc)	~5,49 MB	WAV formátumban, tömörítetlen

2.2.2. A jelek előkészítése: Keretezés és Ablakozás

A folytonos audio streamet a jelfeldolgozáshoz és a statisztikai jellemzők kiszámításához rövid, általában kettő hatványának megfelelő méretű (például 512 vagy 1024 mintából álló) ablakokra, azaz keretekre (frames) bontjuk. Az analizálandó zenei események kerethatáron történő elvesztésének elkerülése végett a kereteket gyakran átfedéssel, egy meghatározott eltolási lépésközz (hop length) alkalmazásával toljuk el egymáson.

Mivel a mesterségesen levágott digitális keretek széleinél ugrásszerű megszakadások keletkeznek, amelyek a frekvenciaelemzés során hamis, magas frekvenciájú *spektrális szivárgást* (spectral leakage) okoznak, a jelekre ablakozó függvényeket (például Hann-ablak) alkalmazunk, amelyek a keretek széleit nullába futtatva "elsimítják" azokat. A 2.2.1. ábra szemlélteti ezt a folyamatot.



2.2.1. ábra: A keretezés (framing) és az ablakozás (windowing) folyamata: a jelet átfedő keretekre bontjuk, majd Hann-ablakot alkalmazunk a spektrális szivárgás elkerülése érdekében

2.2.3. Időtartománybeli jellemzők

Az időtartománybeli (time-domain) jellemzőket közvetlenül a hang hullámformájából (waveform) nyerjük ki, és keretenként számítjuk ki őket. Bár a modern mélytanulás kevésbé támaszkodik rájuk, ezek az értékek sokat elárulnak a különböző zenei műfajok és hangszercsaládok akusztikai eltéréseiről.

- **Amplitúdó burkológörbe (Amplitude Envelope):** A kereten belüli maximális amplitúdóértékekből álló burkológörbe. Jól mutatja a hangok kezdetét (onset), a felfutás (attack) és a lecsengés (decay) idejét.
- **Gyökátlagos négyzetes energia (RMSE):** A keret összes mintájának négyzetes átlagából vont gyök. Sokkal hűebben tükrözi az ember által érzékelt hangerőt (loudness), és jóval kevésbé érzékeny a kiugró, hibás mintákra (outlierekre), mint a burkológörbe.
- **Nullátmenet-gyakoriság (Zero-Crossing Rate – ZCR):** A jel amplitúdójának előjelváltásait számlálja az időtengelyen egy adott kereten belül. Minél gyorsabban oszcillál, azaz minél zajosabb vagy ütősebb a hang, annál magasabb a ZCR értéke. Ez a metrika a gyakorlatban kiválóan elkülöníti a zöngés, tiszta harmonikus hangokat (például az éneket vagy a zongorát, ahol a ZCR alacsony) az erős zaj-komponenssel rendelkező vagy zöngétlen hangoktól.

2.2.4. Hangszercsaládok akusztikai indoklása és hullámformái

A hangszerfelismerő rendszer nem egyedi hangszereket, hanem **hangszercsaládokat** különít el. Ez a döntés nem önkényes, hanem az akusztika fizikájában gyökerezik: az egy családba tartozó hangszerek – azonos hangkeltési elvük és hasonló testszerkezetük miatt – rendkívül hasonló hullámformákat és spektrális lenyomatokat produkálnak, amelyeket a neurális hálózatok is azonos mintázatokként azonosítanak.

A 2.2.2. táblázat összefoglalja a vizsgált hangszercsaládokat és azok jellemzőit.

2.2.2. táblázat: A rendszerben használt hangszercsaládok, a hangkeltés mechanizmusa és a hullámforma jellemzői

Osztály	Hangszerek	Hangkeltés elve	Hullámforma jellemzői
Gitár	Akusztikus és elektromos gitár	Húr pengetése	Gyors, éles tranziens (attack), exponenciális lecsengés (decay). A húr szabadon rezeg.
			
Zongora	Zongora, piano	e- Húr leütése kalapáccsal	Nagyon éles, impulzuszerű tranziens, majd hosszú, fokozatos lecsengés.
			
Vokál	Énekhang, rap, torokének	Hangszálak rezgése	Kváziperiodikus, folytonos hullámforma. Formánsok (a vokális traktus rezonanciái) egyedi spektrális ujjlenyomatot adnak.
			
Vonósok	Hegedű, brácsa, cselló, nagybőgő	Vonó-húr súrlódás	Kitartott, egyenletes amplitúdó mindaddig, amíg a vonó mozog. Csak a test mérete tér el, ami a felhangok relatív intenzitását befolyásolja (nem a burkoló formáját).
			
Nádfúvósok	Klarinét, szaxofon, oboa, fagott	Nád rezgése	Folytonos, kitartott hullámforma. A nád típusa (egy- vs. két-nádas) módosítja a páratlan felhangok arányát, de a burkológörbe azonos jellegű.
			

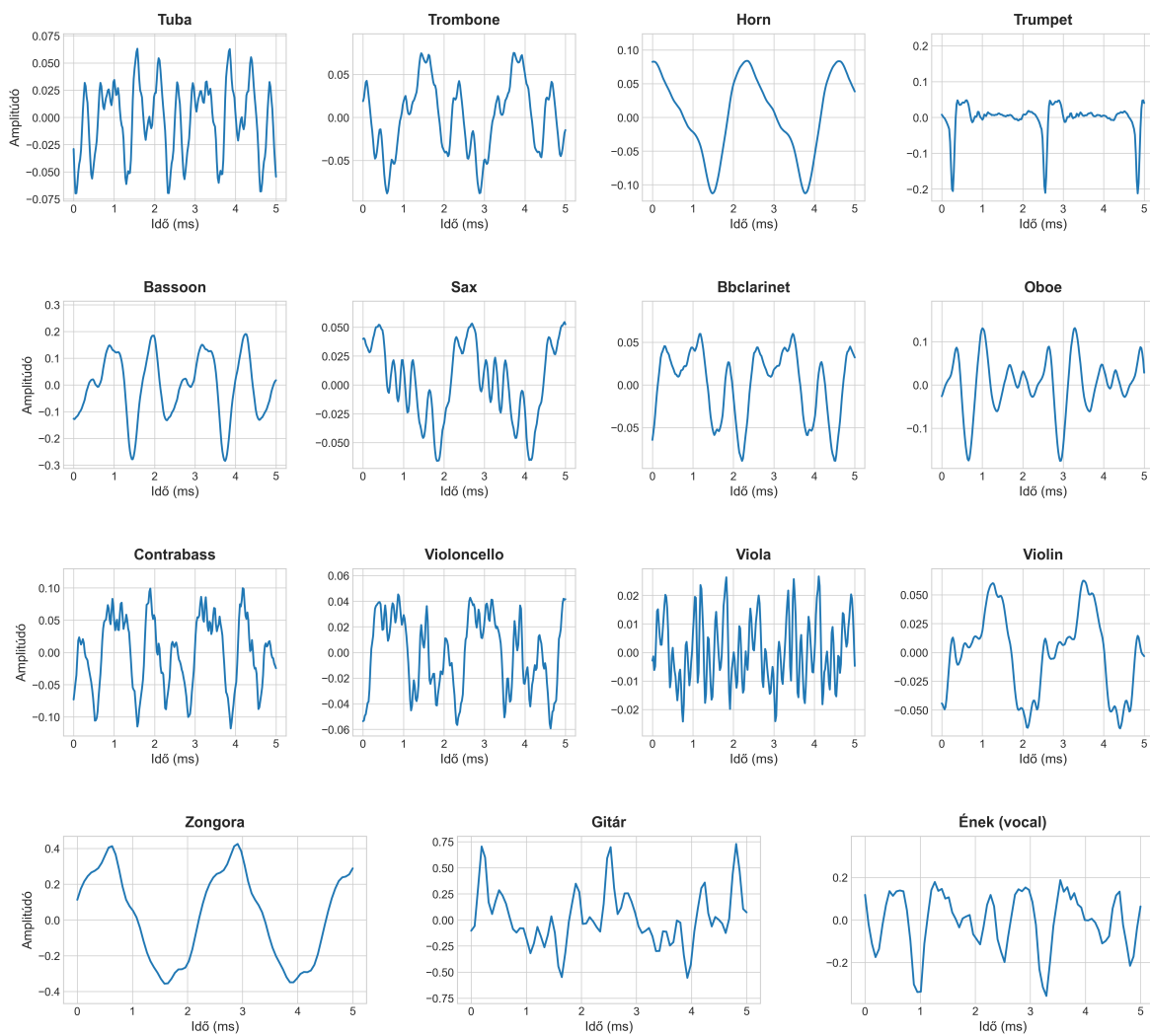
táblázat 2.2.2 – Folytatás az előző oldalról

Osztály	Hangszerek	Hangkeltés elve	Hullámforma jellemzői
Rézfúvósok	Trombita, harsona, kürt, tuba	Ajka rezgése fúvókán	Folytonos, erőteljes amplitúdó, intenzív felharmonikusok. A felhangszerkezet az ajka feszességétől függ.
			
Zaj/Csend	Környezeti zajok, szél, forgalom	Véletlenszerű rezgés	Aperiodikus, rendszertelen hullámforma; nincs domináns frekvenciakomponens.
			

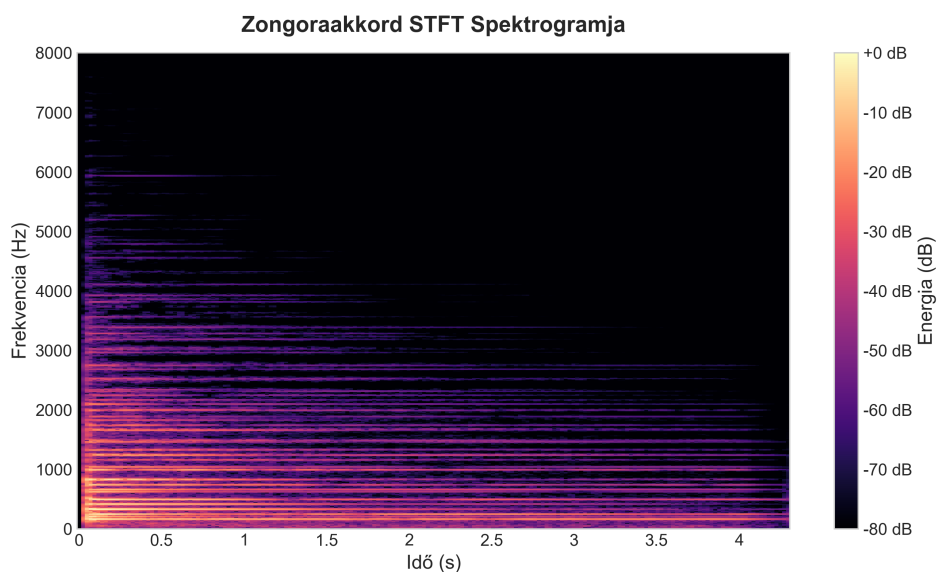
Az egy osztályba tartozó hangszerek periodikus hullámformáinak mikroszkopikus hasonlóságát az alábbi ábra részletesen is szemlélteti. Fontos megjegyezni, hogy a hangszín (timbre) akusztikai definíciója szerint az a tulajdonság, amely alapján két azonos hangmagasságú és hangerejű hangot megkülönböztetünk. Ennek megfelelően, a vizuális összehasonlíthatóság és a „laboratóriumi tisztaság” elemzés érdekében az összes ábrázolt hangszermintát algoritmikusan egy közös alaphangmagasságra (440 Hz, A4) transzponáltuk. Így a periódusidők okozta vizuális különbségek megszűnnek, és az ábrán látható burkológörbék kizárólag a hangszerek testéből és hangképzéséből fakadó egyedi felharmonikus-karakterisztikát mutatják.

2.2.5. Frekvenciatartománybeli jellemzők

Mivel a pusztán időtartománybeli adatok elveszítik a hang pontos frekvenciakomponenseinek eloszlását, nem tudnak eleget a hangszerek egyedi hangszínéről (timbre). Ennek vizsgálatához a jeleket a **Rövid idejű Fourier-transzformáció (STFT)** segítségével frekvenciatartományba konvertáljuk. Az STFT algoritmus az összetett hangokat egyszerű szinuszhullámokra bontja, eredményül pedig egy olyan spektrogramot kapunk, amely egy 2D-s hőképpén (idő és frekvencia) mutatja meg a komponensek energiájának változását.



2.2.2. ábra: A rendszerben használt főbb osztályok mikroszkopikus (5 ms-os) periodikus hullámformáinak összehasonlítása. A vizuális konzisztencia érdekében minden hangszermintát azonos, 440 Hz-es (A4) alapfrekvenciára transzponáltunk. Jól megfigyelhető, hogy a periódusidők kiegyenlítése után az egy osztályba tartozó hangszerek hullámformájának alapszerkezete mekkora hasonlóságot mutat.



2.2.3. ábra: Egy zongoraakkord STFT-alapú spektrogramja: a vízszintes tengely az időt, a függőleges tengely a frekvenciát, a szín intenzitása pedig az energiát (dB) jelöli

A legfontosabb spektrális metrikák:

- **Sávenergia-arány (Band Energy Ratio – BER):** Egy előre meghatározott vágási frekvencia alatti és feletti frekvenciasávokban lévő energia (teljesítmény) arányát fejezi ki, vagyis megmutatja, mennyire dominánsak a mély hangok a vizsgált mintában.
- **Spektrális súlypont (Spectral Centroid):** A magnitúdóspektrum frekvenciáinak intenzitással súlyozott átlaga (a hang energiájának súlypontja). Az emberi fül ezt a hangszín *világosságaként* (brightness) érzékeli: a rezes, éles hangszerek (pl. rézfúvósok) súlypontja magasan van, míg a tompább akusztikus hangszereké lejjebb.
- **Spektrális sávszélesség (Spectral Bandwidth):** A súlypont körüli energiaeioszlás kiterjedése (szórása). Szűk sávszélesség egy tiszta, koncentrált hangot jelöl (pl. szinuszhullám), míg széles sávszélesség zajos, szétszórt spektrumra utal.

2.2.6. Percepció jellemzők és a Mel-spektrogram

A modern gépi tanulásban a modellek pontossága nagyságrendekkel javul, ha a betáplált adatokat az emberi hallás pszichoakusztikai sajátosságaihoz igazítjuk. Az emberi fül sem a hangerőt, sem a frekvenciákat nem lineárisan érzékeli.

A hangosság (loudness) modellálása érdekében az STFT kiszámítása után a kapott lineáris amplitúdó-értékeket egy logaritmikus decibel (dB) skálára konvertáljuk. Ugyanakkor a frekvencia-érzékelésünk is torzított: az alacsony frekvenciák közötti eltéréseket (pl. 200 Hz és 400 Hz között) zeneileg markánsnak halljuk, míg a magas tartományokban az azonos Hz értékű különbségeket (pl. 10 000 Hz és 10 200 Hz) az emberi fül szinte egyáltalán nem képes megkülönböztetni.

Ezt a biológiai sajátosságot hivatott lekövetni a **Mel-skála**, amely az emberi érzékeléshez igazítja a frekvenciatengelyt. Az algoritmus az STFT kimenetére egymást átfedő háromszög-szűrőket (Mel-szűrőbankokat) illeszt, amelyek a mélyebb frekvenciákon sűrűbben, a magasabbakon ritkábban helyezkednek el. Ennek az eljárásnak a végeredménye a **Mel-spektrogram**. A 2.2.4. ábra szemlélteti a lineáris és a Mel-skálájú frekvenciatengely közötti különbséget.



2.2.4. ábra: Egy hangfelvétel lineáris frekvenciaskálájú (felső) és Mel-skálájú (alsó) spektrogramjának összehasonlítása. A Mel-skála az alacsony frekvenciákon finomabb felbontást biztosít, így az ember számára releváns hangmagasság-információt hűbben tükrözi.

Míg korábban a gépi tanulásban a hangszín (timbre) és a beszéd (formánsok) modellezésére a Mel-spektrogram további Diszkrét Koszinusz Transzformációjával (DCT) előállított **MFCC** (Mel-Frequency Cepstral Coefficients) együtthatókat használták, a mai modern Konvolúciós Neurális Hálózatok (CNN) és mélytanulósos (Deep Learning) architektúrák többnyire magát a nyers Mel-spektrogramot alkalmazzák. Az MFCC használata túlzott emberi előítéleteket és adatvesztést jelenthet a hálózat számára. Ezzel szemben a 2D-s képi mátrixként betáplált Mel-spektrogramból a hálózat saját maga tanulja meg kinyerni azokat a komplex rejtett reprezentációkat, amelyek az osztályozáshoz és az eszközök felismeréséhez szükségesek.

A 2.2.3. táblázat összefoglalja a két megközelítés közötti különbségeket.

2.2.3. táblázat: Mel-spektrogram és MFCC összehasonlítása gépi tanulási szempontból

Szempont	MFCC	Mel-spektrogram
Adat típusa	1D koefficiens-vektor (általában 13-40 érték/keret)	2D mátrix (frekvencia $\times$ idő hőtérkép)
Információ-megőrzés	A DCT lépés eldobja a finom spektrális részleteket	Az összes Mel-szűrőbank ki-menetét megőrzi
Emberi torzítás	Erős (formánsokra fókuszál)	Közepes (logaritmikus skálázás)
Modern alkalmazás	Hagyományos ML (SVM, GMM)	Deep Learning (CNN, Transfer Learning)

2.3. Ismert hasonló alkalmazások

2.4. Felhasznált technológiák és eszköztár

A gépi tanuláshoz és az audio jelfeldolgozáshoz (a hullámformák betöltésétől a jellemzők kinyeréséig) szükséges szoftveres folyamatok kivitelezésére a széles körben elterjedt **Python** programozási nyelvet alkalmaztam. A fejlesztési és tesztelési fázisokban interaktív Jupyter Notebookokat is használtam, amelyek megkönnyítették a hangfájlok auditív ellenőrzését és a kód vizualizációját. Az adatfeldolgozási szoftveres *stack* (eszközkészlet) az alábbi főbb könyvtárakra épül:

- **Librosa:** A legfontosabb nyílt forráskódú Python könyvtár a zenei adatok és audio jelek elemzéséhez. Hatékony, beépített függvényekkel (például a `librosa.load`, `librosa.stft`, vagy a `librosa.feature.melspectrogram`) támogatja az időbeli és frekvenciatartománybeli komplex jellemzőkinyerést anélkül, hogy az algoritmusokat nulláról (from scratch) kellene implementálni. Alapértelmezett mintavételezési frekvenciája a gépi elemzéseknél elterjedt 22 050 Hz, emellett a feldolgozás egyszerűsítése érdekében automatikusan monó jellé konvertálja a bemenetet.
- **NumPy:** A hangjelek és spektrogramok memóriában történő tárolásáért felel. Az egydimenziós (nyers hanghullám) és többdimenziós (Mel-spektrogram) számtömbökként (array) történő adatkezelés rendkívül gyors és optimalizált matematikai transzformációkat tesz lehetővé a háttérben.
- **Matplotlib:** A kinyert reprezentációk – hullámformák, burkológörbék, valamint a hőtérképként megjelenő log-mel spektrogramok – vizualizálásának alapvető keretrendszere, amelyet gyakran a Librosa saját, `specshow` nevű megjelenítő moduljával együttesen alkalmazunk a tengelyek (idő és frekvencia) helyes szinkronizálása érdekében.

3. A RENDSZER SPECIFIKÁCIÓI ÉS ARCHITEKTÚRÁJA

A szakdolgozat előző fejezetében bemutatott akusztikai és mélytanulási elméleti alapokra építkezve jelen fejezet a megvalósított hangszerfelismerő rendszer mérnöki és szoftverfejlesztési tervezését ismerteti. Egy gépi tanulási alapú valós idejű hangfelismerő rendszer fejlesztése során nem csupán a neurális hálózat architektúrájára kell fókuszálni, hanem egy olyan robusztus csővezeték (pipeline) kialakítására is, amely képes a nyers adatok betöltésétől kezdve az adatok transzformációján át egészen a végfelhasználói valós idejű interakcióig minden lépést zökkenőmentesen kiszolgálni.

Ez a fejezet definiálja a rendszerrel szemben támasztott funkcionális és nem funkcionális követelményeket, bemutatja a felhasználói eseteket (Use Case), valamint részletezi az alkalmazás belső szoftverarchitektúráját és a technológiai választások hátterét.

3.1. Célkitűzések és követelményelemzés

A fejlesztés fő célkitűzése egy olyan keretrendszer létrehozása volt, amely képes előre felvett zenei adathalmazokon különböző jellemzőkinyerési módszereket alkalmazni, azokkal modern konvolúciós neurális hálózatokat tanítani, majd a legpontosabb modellt egy valós idejű (mikrofonos) környezetben is megbízhatóan futtatni.

3.1.1. Funkcionális követelmények

A funkcionális követelmények azokat a konkrét folyamatokat és szolgáltatásokat írják le, amelyeket a rendszernek feltétlenül nyújtania kell a sikeres működés érdekében:

- **Adathalmazok fúziója és tisztítása:** A szoftvernek képesnek kell lennie nyílt forrású (pl. NSynth, Good-sounds) és saját gyűjtésű adathalmazok egyesítésére, a hibás vagy csendes minták algoritmikus szűrésére.
- **Jellemzőkinyerés (Feature Extraction):** A nyers .wav fájlokból automatikusan generálni kell a spektrogramokat (STFT és Mel-spektrogram) a megadott ablakozási paraméterekkel, és azokat hatékony Numpy mátrix formátumban menteni.
- **Modell tanítása (Training Pipeline):** Egy konfigurálható csővezeték megléte, amely betölti az előkészített adatokat, definiálja a 2D CNN architektúrát, majd korai leállás (early stopping) használatával optimalizálja a modellt, és kimentti a legjobb súlyokat.
- **Valós idejű predikció:** Egy önálló futtatható modul, amely másodpercenként rögzíti a számítógép mikrofonjának jelét, valós időben jellemzőt von ki, és a betanított Keras modellen lefuttatva terminálos felületen jelzi a felismert hangszercsaládot.

3.1.2. Nem funkcionális követelmények

A nem funkcionális követelmények a rendszer működésének minőségi mutatóira fókuszálnak:

- **Alacsony késleltetés (Low Latency):** A valós idejű működés kritikus feltétele, hogy az 1 másodperces audio ablak rögzítése és a predikció megjelenítése között eltelt idő ne haladja meg a 200–300 milliszekundumot, ezáltal a felhasználó azonnali visszajelzést kapjon.
- **Erőforrás-hatékonyság:** A jellemzőkinyerésnek és a predikciónak CPU-n is zökkenőmentesen kell futnia, elkerülve a memóriaszivárgást egy végtelenített valós idejű futás (pl. folyamatos élő koncert figyelése) során.
- **Modularitás:** A kódnak lazán csatolt (loosely coupled) szkriptekből kell állnia, hogy a jellemzőkinyerési algoritmus, a neurális háló architektúrája vagy a mikrofonos rögzítési logika egymástól függetlenül módosítható vagy cserélhető legyen.

3.2. Használati esetek (Use Case modell)

A rendszer interakciós felületeit és főbb felhasználási forgatókönyveit egy UML Use Case diagram (3.2.1. ábra) segítségével modelleztük. Két fő aktort azonosíthatunk:

1. **Felhasználó / Kutató:** Az a mérnök vagy adattudós, aki a modell betanításáért és teszteléséért felelős.
2. **Mikrofon / Bemenet (Rendszer-aktor):** A valós idejű adatáramlást biztosító hardveres periféria, amely automatikusan triggeli a rendszer predikciós eseményeit.

A használati esetek világosan elkülönítik a "laboratóriumi" vagy kutatási fázist (adat-előkészítés és tanítás) a "produkciós" fázistól (élő predikció). Fontos tervezési szempont volt, hogy a valós idejű hangfelvétel folyamata szorosan inkluzív («*include*») kapcsolatban álljon a hangszer predikciójával, hiszen a mikrofon bufferének megtelése azonnal és megszakítás nélkül aktiválja az osztályozó modult.

3.3. Rendszerarchitektúra és adatfolyam

Egy robusztus gépi tanulási rendszer architektúrája túlmutat magán a neurális hálózaton; magában foglalja az adatbeviteli (ingestion), az adatfeldolgozási (preprocessing) és az eredmény-megjelenítési (serving) rétegeket is. A megvalósított rendszer szoftverarchitektúráját és az adatok útját a 3.3.1. ábra UML komponensdiagramja vizualizálja.

Az architektúra négy logikailag szeparált csomópontra (csomagra) bontható:

1. **Környezet és Bemenet:** Felelős az adatok biztosításáért. Offline fázisban a .wav és .mp3 fájlokból álló hatalmas adathalmazokat biztosítja lemezzről, míg online fázisban a rendszer közvetlen I/O kapcsolatot tart a számítógép hangkártyájával.

An error has occurred : java.lang.NoSuchFieldError: java.awt.image.Kernel.width
I saw a glimpse of my future and everything's changed for me now.



PlantUML (1.2026.1) has crashed.

Diagram size: 27 lines / 622 characters.

Java Runtime: GraalVM Runtime Environment
JVM: Substrate VM
Default Encoding: UTF-8
Language: null
Country: null

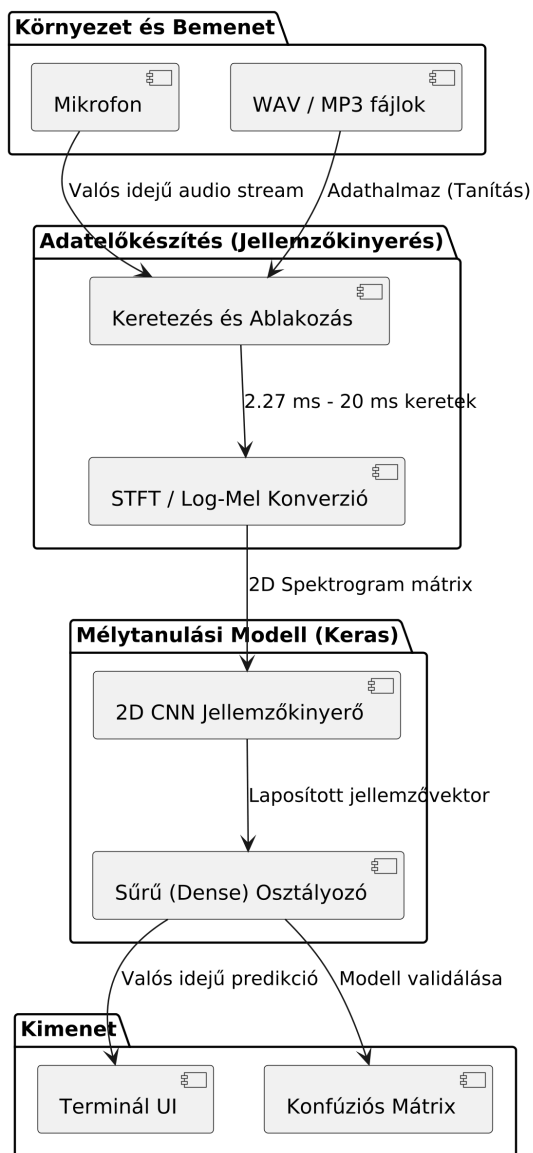
PLANTUML_LIMIT_SIZE: 4096

You should send this diagram and this image to **plantuml@gmail.com** or
post to **<https://plantuml.com/qa>** to solve this issue.
You can try to turn around this issue by simplifying your diagram.

```
java.lang.NoSuchFieldError: java.awt.image.Kernel.width
org.graalvm.nativeimage.builder/com.oracle.svm.core.jni.functions.JNIFunctions$Support.getFieldID(JNIFunctions.java:1864)
org.graalvm.nativeimage.builder/com.oracle.svm.core.jni.functions.JNIFunctions.getFieldID(JNIFunctions.java:461)
java.desktop@21.0.10/java.awt.image.Kernel.initIDs(Native Method)
java.desktop@21.0.10/java.awt.image.Kernel.<clinit>(Kernel.java:49)
net.sourceforge.plantuml.klimt.drawing.g2d.DriverShadowedG2d.getConvolveOp(DriverShadowedG2d.java:58)
net.sourceforge.plantuml.klimt.drawing.g2d.DriverShadowedG2d.drawOnlyLineShadowSpecial(DriverShadowedG2d.java:165)
net.sourceforge.plantuml.klimt.drawing.g2d.DriverRectangleG2d.draw(DriverRectangleG2d.java:87)
net.sourceforge.plantuml.klimt.drawing.g2d.DriverRectangleG2d.draw(DriverRectangleG2d.java:59)
net.sourceforge.plantuml.klimt.drawing.AbstractUGraphic.draw(AbstractUGraphic.java:144)
net.sourceforge.plantuml.decoration.symbol.UsymbolRectangle.drawRect(UsymbolRectangle.java:70)
net.sourceforge.plantuml.decoration.symbol.UsymbolRectangle.access$000(UsymbolRectangle.java:52)
net.sourceforge.plantuml.decoration.symbol.UsymbolRectangle$2.drawU(UsymbolRectangle.java:109)
net.sourceforge.plantuml.svek.ClusterDecoration.drawU(ClusterDecoration.java:78)
net.sourceforge.plantuml.svek.Cluster.drawU(Cluster.java:372)
net.sourceforge.plantuml.svek.SvekResult.drawU(SvekResult.java:74)
net.atmp.ImageBuilder.writeImageInternal(ImageBuilder.java:275)
net.atmp.ImageBuilder.write(ImageBuilder.java:239)
net.sourceforge.plantuml.svek.CucaDiagramFileMakerSvek.createFileInternal(CucaDiagramFileMakerSvek.java:142)
net.sourceforge.plantuml.svek.CucaDiagramFileMakerSvek.createFile(CucaDiagramFileMakerSvek.java:70)
net.atmp.CucaDiagram.exportDiagramInternal(CucaDiagram.java:489)
net.sourceforge.plantuml.UmlDiagram.exportDiagramNow(UmlDiagram.java:119)
net.sourceforge.plantuml.AbstractPSystem.exportDiagram(AbstractPSystem.java:216)
net.sourceforge.plantuml.SourceStringReader.outputImage(SourceStringReader.java:189)
net.sourceforge.plantuml.Pipe.generateDiagram(Pipe.java:108)
net.sourceforge.plantuml.Pipe.managePipe(Pipe.java:99)
net.sourceforge.plantuml.Run.main(Run.java:182)
java.base@21.0.10/java.lang.invoke.LambdaForm$DMH/sa346b79c.invokeStaticInit(LambdaForm$DMH)
```

3.2.1. ábra: A hangszerfelismerő rendszer főbb használati esetei (UML Use Case diagram)

Please use 'option handwritten true' to enable handwritten



3.3.1. ábra: A rendszerarchitektúra és a folyamatos adatfolyam (UML Komponensdiagram)

2. **Adatelőkészítés (Jellemzőkinyerés):** Ez a modul a rendszer "szíve" az akusztikai feldolgozás szempontjából. A beérkező folytonos hanghullámokat átfedő keretekre (framing) és ablakokra vágja, majd elvégzi a Fourier-transzformációt (STFT) és a Mel-szűrőbankos konverziót. Kimenete mindig egy fix dimenziójú 2D mátrix, amely már skálázva, normalizálva áll készen a modell számára.
3. **Mélytanulási Modell (Keras):** A 2D konvolúciós rétegekből felépülő jellemzőkinyerő, majd az azt követő Sűrű (Dense) osztályozó hálózat. Bemenete a képszerű spektrogram, kimenete pedig egy valószínűségi vektor, amely a hangszer-osztályok eloszlását mutatja.
4. **Kimenet:** A fejlesztői terminálon (CLI) megvalósított frissülő felület, amely előben listázza a predikciós százalékokat, valamint az offline validációs fázisban itt generálódnak a konfúziós mátrixok és a tanulási görbék.

3.4. Alkalmazott technológiák és fejlesztői környezet

A projekt megvalósítása során kiemelt szempont volt a nyílt forráskódú, ipari szabványnak számító eszközök használata, amelyek kiterjedt dokumentációval és tudományos elfogadottsággal rendelkeznek. A fejlesztés Mac OS és Windows operációs rendszereken egyaránt tesztelve lett, biztosítva a platformfüggetlenséget.

3.4.1. Programozási nyelv és alapkörnyezet

A rendszer gerincét a **Python** programozási nyelv adja. A Python választását annak egyedülálló adattudományi és gépi tanulási ökoszisztémája, valamint a gyors prototipizálást támogató absztrakciói indokolták. A numerikus számításokat, a mátrix-műveleteket és a spektrogramok memóriahatékony tárolását a **NumPy** könyvtár biztosítja, míg az adatok struktúrált elemzését és tisztítását a **Pandas** adatkeretek (DataFrames) végzik. A valós idejű, alacsony késleltetésű mikrofonos rögzítést a **sounddevice** C-alapú Python wrapper hívásai teszik lehetővé.

3.4.2. Akusztikai jelfeldolgozás: Librosa

A hangjelek beolvasásához, csendek levágásához, és a komplex idő-frekvencia transzformációk (STFT, Mel-spektrogram, MFCC) elvégzéséhez a **Librosa** könyvtárt alkalmaztam. A Librosa a zenei és audio elemzés (MIR) legelismertebb Python eszköze, amely natívan támogatja a pszichoakusztikai skálákat (például a Mel-skálát), így drasztikusan lerövidíti az akusztikai algoritmusok implementációs idejét, garantálva a laboratóriumi pontosságot.

3.4.3. Mélytanulási keretrendszer: TensorFlow és Keras

A mesterséges neurális hálózat architektúrájának definícióját, a modell tanítását és a valós idejű predikciók inferenciáját a **TensorFlow** platformon futó **Keras** API végzi. A

Keras választása a rendkívül gyors fejlesztési ciklus, a kényelmes réteg-absztrakciók (Conv2D, MaxPooling, Dropout) és a könnyű menthetőség miatt történt. A modell betanítása és az adathalmaz validálása nagy kapacitású hardveres környezetben futott, de az inferencia (az élő felismerés) során az elkészült hálózat olyan mértékben optimalizált maradt, hogy egy átlagos számítógép processzorán (CPU) is akadás nélkül, töredékmásodperces válaszidővel képes működni.

4. RÉSZLETES TERVEZÉS

A következő fejezetben részletes betekintés nyújtok, organikus adathalmazzal rendelkező, saját célú hangszerfelismerő rendszer felépítésébe, ismertetve minden tervezési lépést. Ez többek között tartalmazza az adatkészlet létrehozását, a jellemzési paramétereket, az adatbővítési beállításokat és a 2D konvolúciós neurális hálózatokat, a YAMNet transzfer tanulási architektúrát, végül a valós idejű hangszerfelismerő modul tervezését.

4.1. Az adathalmaz tervezése

Az adathalmaz szigorú szempontrendszer szerint került összeállításra, különös figyelmet szentelve a változatos és felismerhető hangszerjátéokra.

Az osztályrendszert hét osztályra osztottuk, amelyek a könnyű és klasszikus zenében előforduló leggyakoribb hangszertípusokat fogják reprezentálni. Van egy extra, környezeti zaj osztály is, amely segítségével kezeljük a csendet és a váratlan hangzavart. A 7 osztály a következő:

- (1) Gitár: tartalmazza az akusztikus és a kevésbé torzított elektromos gitárokat is
- (2) Zongora: tartalmazza a klasszikus és digitális zongorák egyes típusait
- (3) Vokál: tartalmaz emberi éneklést, rapet és egyéb sajátos hangképzéseket (mongol torokének, szájdob)
- (4) Húr: a vonós négyes hangszerei: hegedű, brácsa, cselló, bőgő
- (5) Nádfúvósok: mind egynádas (klarinét, szaxofon), mind kétnádas (fagott, oboa)
- (6) Rézfúvósok: legelterjedtebb változatok: trombita, harsona, kürt, tuba
- (7) Zaj: Háttér- és/vagy környezeti zajok (szél, eső, forgalom stb.), madárcsicsergés stb.

4.1.1. Saját készítésű organikus adathalmaz

Kezdetben a létező adathalmazokat vizsgáltam, hogy megértsem, hogyan érdemes elkezdeni a tervezési folyamatot, és megfigyeljem, miben erősek külön-külön. Ennek érdekében az előkészítő szakaszban lefuttattam egy elemző szkriptet (`01_analyze_datasets.py`), amely feltárta az elérhető források struktúráját. Az elemzés eredményeit a 4.1.1. táblázat foglalja össze.

4.1.1. táblázat: A vizsgált publikus adathalmazok összehasonlító elemzése

Adathalmaz	Tartalom leírása	Fő erősségek	Korlátok / Miért nem felelt meg
IRMAS [2]	Zenei részletek (3 s) szóló hangszeres és polifón szekciókkal	Valós zenei környezet, polifónia vizsgálata	Korlátozott számú és egyenetlen eloszlású hangszerkategóriák.
NSynth [3]	300 000+ db egyedi hangszeres hangjegy (4 s)	Hatalmas mintaállomány, egységes hanghossz és pitch	Csak izolált hangok, nincs benne valós zenei folytonosság és frazeálás.
Medley-solos-DB [4]	Különböző sávokból ki-nyert szóló hangszeres részek (3 s)	Valós zenei előadásokból származó, tiszta szóló felvételek	Alacsony mintaszám egyes kategóriákban, szűkös osztályválaszték.
TinySOL [5]	Izolált zenekari hangszerek hangjai, különböző technikákkal	Kiváló stúdióakusztika, gazdag játéktechnikai annotációk	Kizárólag izolált hangjegyek, nem reprezentálja a folyamatos zenélést.
Good-sounds [6]	Fúvós és vonós hangszerek egyedi hangjegyei	Különböző minőségű mikrofonok és felvételi minőségek	Nagyon szűk hangszerpaletta (pl. nincs gitár vagy énekhang).
ESC-50 [7]	2000 db környezeti és háttérzaj minta (5 s)	50 különböző zajkategória rendkívül széles spektruma	Nem tartalmaz zenei hangszereket, csak a zaj kategóriához alkalmas.

Megállapítottam, hogy a meglévő adathalmazok teljes vagy részleges felhasználása nem felel meg az igényeimnek. Minden adathalmaz mintái eltérő hosszúságúak, más a hangszerosztályok eloszlása, rengeteg mintát tartalmaznak, és különböző céllal jöttek létre. Emiatt – bár kiváló tervezési alapul szolgáltak – a feldolgozásuk egységesítése komplex feladat lenne, és az átválogatásuk is rengeteg időt igényelne. Egy organikus, a saját elképzeléseim alapján összeállított adathalmaz biztosabb és átláthatóbb alapokat nyújt egy jól működő modell megteremtéséhez. Ez alól kivételt képezett az ESC-50, amely sokféle környezeti és háttérzaj-mintát foglal magába, így hiánypótlóként szolgált egy mindenre kiterjedő zajos adathalmaz összeállításához.

Az interneten található zenei felvételekből válogattam, figyelmet fordítva az aktuális hangszerek karakterességének kiemelésére és a zenei játék változatosságára. Minden hangszerosztályhoz (gitár, zongora, ének, vonós, nád- és rézfúvós) nagyjából 100–150 darab interneten elérhető felvételt választottam ki, amelyekből az eredeti forrás zenei tartalmának függvényében

5 másodperces blokkokat vágtam ki.

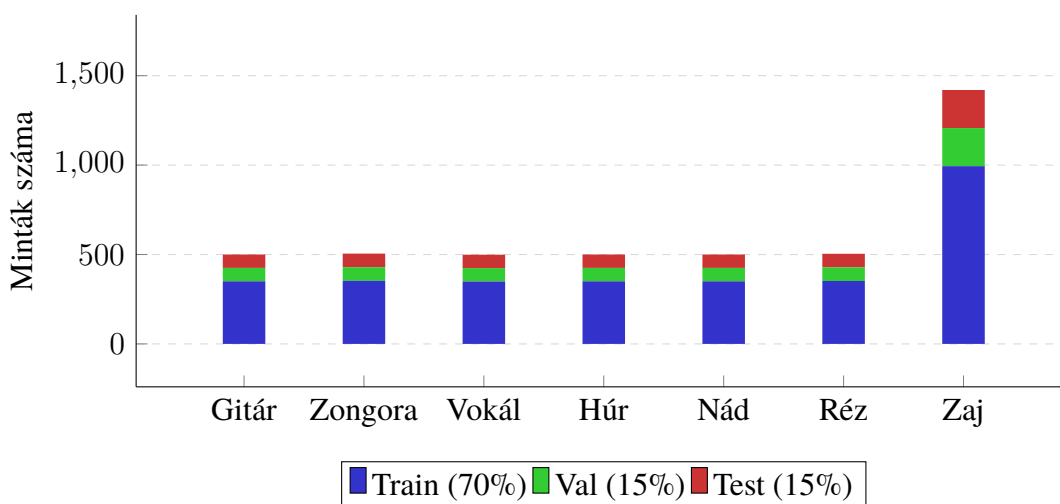
Ezen a ponton egy kritikus tervezési döntést hoztam a gépi tanulásban gyakran felmerülő adatszivárgás (data leakage) elkerülése érdekében. Ha a darabolást rögtön végrehajtom, majd az összes 1 másodperces szeletet utólag osztom szét, nagy az esélye annak, hogy ugyanannak a hangszernek ugyanabból a zenei frázisából kerüljenek minták mind a tanító (train), mind a validációs (val) vagy teszt (test) halmazba. Ez túlzottan optimista, hamis eredménybecsléshez (túltanuláshoz, azaz overfittinghez) vezetne, mivel a modell szinte azonos hangokat látna a teszteléskor, mint a tanításkor.

Ennek elkerülésére a `01_prepare_clean_dataset.py` szkript segítségével először a teljes, 5 másodperces blokkokat kevertem meg és osztottam fel a tanító (70%), validációs (15%) és tesztelő (15%) halmazokra. Ezt a műveletet úgynevezett **rétegzett (stratified) felosztással** végeztem el, ami garantálja, hogy a szkript minden egyes hangszerosztálynál önállóan és szigorúan betartsa ezt az arányt, így az osztályok eloszlása mindhárom halmazban (train, val, test) tökéletesen megegyező és kiegyensúlyozott marad. A szkript csak ezután – a már szigorúan szeparált blokkokat – szeletelte fel 1 másodperces, átfedésmentes klipekre, megőrizve a szeparáció integritását. Ezáltal biztosítható, hogy a kiértékelési halmazok (val/test) eredményei teljesen megbízhatóak, és a modell nem tud „csalni”.

Ez a folyamat hangszertípusonként nagyjából 500 darab, többnyire monofón, stúdió- és szobaminőségű mintát eredményezett. A teljes tiszta adathalmaz (beleértve az ESC-50-ből származó kiterjedt zaj osztályt is) így összesen 4428 mintából áll.

Fontosnak tartom hangsúlyozni, hogy az internetről gyűjtött zenei hangmintákat kizárólag kutatási és oktatási (akadémiai) célokra használok fel. Ezeket a mintákat önmagukban nem teszem közzé, nem terjesztem nyilvánosan, és semmilyen kereskedelmi vagy pénzszerzési célra sem használok fel. Ez a megközelítés teljesen analóg a nagy technológiai vállalatok által alkalmazott gyakorlattal, ahol a nyilvánosan elérhető adatokat szigorúan a modellek tanítására használják fel (a szabad felhasználás, azaz a *fair use* elve alapján), miközben maguk a nyers forrásfájlok zártak maradnak.

Az adathalmaz eloszlását ezen lépés után a 4.1.1. ábra szemlélteti.



4.1.1. ábra: A saját gyűjtésű organikus adathalmaz eloszlása a szeparált szeletelés után

4.1.2. Mikrofonos újrafelvétel a valós idejű kiértékelés felkészítésének érdekében

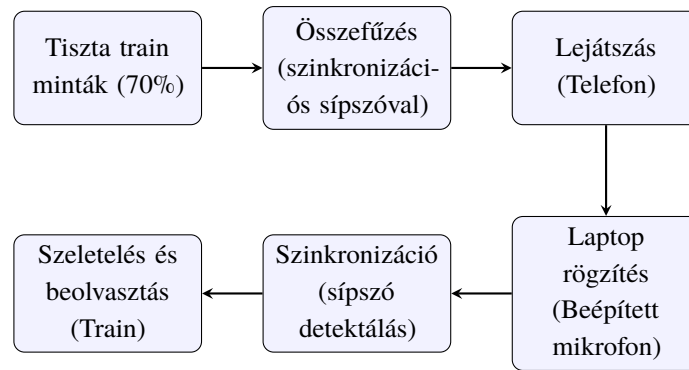
A kezdeti adatkészleteim önmagukban kevés zajos mintát tartalmaznak, melyeket az eredeti felvételt készítő eszköz mikrofonjával rögzítettek. Minden mikrofonnak saját jellemzői vannak: frekvencia-válasza, érzékenysége, torzítása, belső zaja és maximális hangnyomásszintje (SPL). Mivel a projektem célja a valós idejű hangszerfelismerés is, a modell stabilitásának kialakításához elengedhetetlen az optimalizálás a saját eszközünkre, amellyel a predikció megtörténik.

A valós idejű kiértékelésre való felkészítés érdekében egy speciális eljárást találtam ki. Mivel nem férek hozzá, és játszani sem tudok ennyiféle hangszeren, a telefon hangszóróján játszom vissza a mintákat a laptopom beépített mikrofonjába. A feljátszás 10–30 cm távolságról történik a beépített mikrofon pontos helyzetétől mérve, hogy a hang a közelség miatt se legyen túl torz, de a távolság miatt se váljon értelmezhetetlenné vagy túl halkká.

A több ezer minta egyenkénti feljátszása helyett egy automatizált munkafolyamatot alakítottam ki. Egyetlen közös script (`02_acquire_mic_data.py`) segítségével először osztályonként összefűzöm a tiszta adatok 70%-át kitevő tanító (train) halmaz mintáit, a telefonról visszajátszva egyben felvételezem, majd a szoftver automatikusan újra felszeleteli azokat. Ezt követően egy dedikált script (`03a_merge_mic_to_train.py`) végzi el az új mikrofonos minták célirányos fúzióját a tanító adathalmazzal. Kritikus tervezési döntés, hogy ezt a mikrofonos újrafelvételt kizárólag a tanító halmazon végzem el, míg a validációs és teszt halmazok teljesen „tiszták” maradnak. Ezzel elkerülhető az adatszivárgás (data leakage): ha a modell a tesztelés során a saját mikrofonunk már megismert alapzajával és torzításával találkozna, a specifikus zajokat felismerve „csalhatna”. A tiszta kiértékelő halmazok garantálják, hogy a modell valós generalizációs képességét mérjük egy számára teljesen ismeretlen akusztikai környezetben.

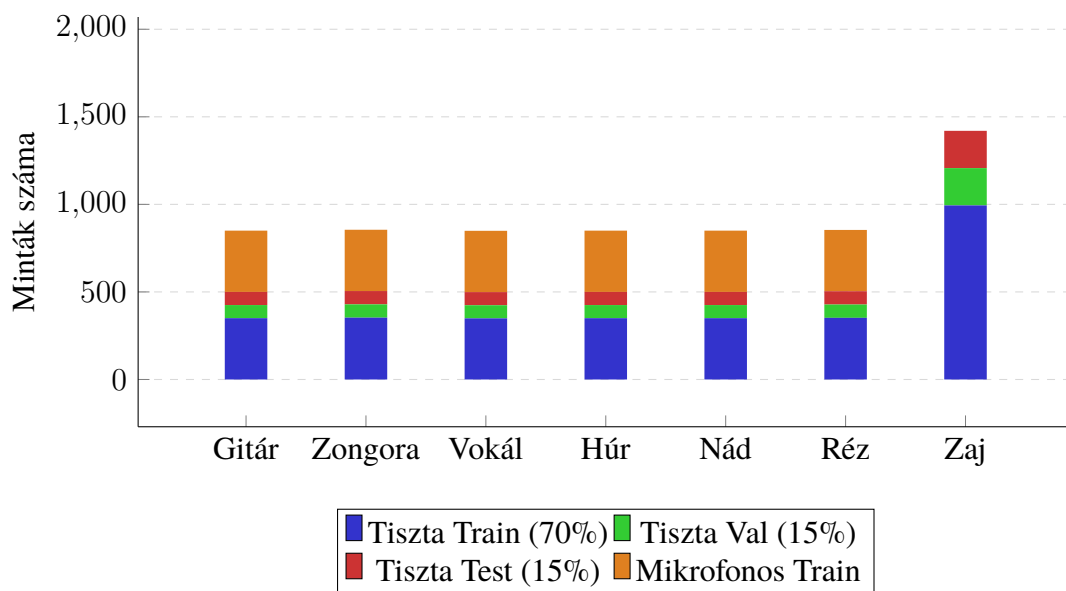
Ebben az esetben azonban a telefon lejátszásának és a laptop beépített mikrofonjának felvételezésének szinkronizációjára van szükség. A pontos szinkronizáció megvalósításához a következő folyamatot alakítottam ki: a `02_acquire_mic_data.py` script első előkészítő lépése a hanganyag legelejére egy 1 másodperces, 440 Hz-es szinuszos szinkronizációs sípszót generál, a hangszeres mintákat pedig 0,2 másodperces csendes szakaszokkal választja el egymástól. A telefonról történő visszajátszás alatt ugyanez a script rögzíti a beérkező jelet 16 000 Hz-es mintavételezéssel, mono formátumban. Végül a script szeletelő funkciója a Librosa könyvtár segítségével detektálja a felvétel elején található sípszó végét, majd ettől a ponttól kiindulva felszeleteli a hanganyagot pontosan 1 másodperces egységekre, miközben automatikusan kiszűri a csendes részeket.

Az automatizált mikrofonos újrafelvételi és szeletelési munkafolyamatot a 4.1.2. ábra szemlélteti.



4.1.2. ábra: A mikrofonos újrafelvételi és szinkronizált szeletelési folyamat lépései

Az újrafelvételi folyamat eredményeképpen kizárólag a tanító (train) halmaz eredeti, tiszta mintái mellé kerülnek be az új, mikrofonos karakterisztikájú minták. Ez a bővítmény hordozza a laptopom beépített mikrofonjának torzításait, frekvencia-választát és belső zaját, felkészítve ezzel a modellt a valós idejű kiértékelés kihívásaira. Az adathalmaz aktuális összetételét és a minták felhalmozódását a 4.1.3. ábra szemlélteti egymásra épülő (stacked) formában, halmazok (Train, Val, Test) és rögzítési mód (Tiszta, Mikrofonos) szerint különválasztva. Fontos megjegyezni, hogy a Zaj osztály esetében nem végeztem ilyen újrafelvételt: az ebből a kategóriából származó ESC-50 minták eleve valós környezeti és háttérzajokat tartalmaznak, így teljesen feleslegesnek tartottam a szélzaj vagy az eső hangját telefonról visszajátszva újra felvenni, ráadásul ez a lejátszási lánc miatt szükségtelen egyedi torzulásokat is vinne a tiszta zajmintákba.



4.1.3. ábra: A saját tiszta és az újrafelvelezett zajos minták megoszlása az adathalmazban

4.1.3. Végző adathalmaz összeállítása

Egy konzisztens és pontos adathalmaz nagyban segíti a neurális hálózat tanulási folyamatát. A megtervezett adatfolyam két fő könyvtárra támaszkodik: a `dataset_clean`

(amely a tiszta internetes mintákat tartalmazza, összesen 4428 darabot) és a `dataset_mic` (amely a `train` halmaz esetében már tartalmazza az újrafelvett, mikrofonos mintákat is, így összesen 6492 darabot számol).

Amikor a jellemzőkinyerő script lefut, az adataugmentációs lépéseket (reverb, zajkeverés, torzítás) „on-the-fly”, azaz dinamikusan végzi el a memóriában. Ennek eredményeképpen a feldolgozott (kinyert) jellemzők száma a tanítási fázishoz kibővül: a tiszta adathalmazban 5927-re, míg a mikrofonos (augmented) adathalmazban 8335-re. Ez a megközelítés rendkívül stabil alapokat biztosít a hálózat számára.

4.2. Jellemzőkinyerés tervezése

A nyers audiojelek nem alkalmasak közvetlenül a konvolúciós neurális hálózat bemeneteként, mivel a hullámforma (waveform) időtartománybeli ábrázolása túl magas dimenziójú és nehezen tanulható mintázatokot tartalmaz. Emiatt szükséges egy megfelelő 2D-s spektrális jellemzőkinyerési eljárás kiválasztása. A tervezés során három elterjedt módszert vizsgáltam meg a `librosa` [8] könyvtár implementációira építve: a rövid idejű Fourier-transzformációt (STFT), a Mel-frekvenciás kepsztrális együttthatókat (MFCC), valamint a Log-Mel spektrogramot. Mindhárom jellemzőtípus azonos ablakozási paraméterekkel készül:

- FFT ablakméret: $n_{fft} = 1024$ minta,
- Ugrási hossz (hop length): $hop_length = 512$ minta,
- Mintavételi frekvencia: $SR = 16\,000$ Hz.

1 másodperces audioszelet esetén ($16\,000$ minta, $hop_length = 512$) az időtengely mérete: $\lfloor 16000/512 \rfloor + 1 = 32$ időkeret.

4.2.1. Log-Mel spektrogram

A Log-Mel spektrogram előállításának lépései:

1. Mel-szűrőbank alkalmazása $n_{mels} = 64$ sávval az STFT kimenetére,
2. Teljesítmény-decibelre konvertálás: $S_{dB} = 10 \cdot \log_{10}(S_{mel}/S_{ref})$.

Az eredmény egy 64×32 méretű mátrix, amelyet a CNN egysatornás képként ($64 \times 32 \times 1$) kap meg. A Log-Mel spektrogram előnye, hogy a Mel-skála a hallórendszer nemlineáris frekvenciaérzékelését modellezi, így a hangszerek jellegzetes formánsai és felharmonikusai hatékonyan jelennek meg.

4.2.2. STFT spektrogram

Az STFT (Short-Time Fourier Transform) spektrogram az amplitúdó-spektrum decibelre konvertált változata:

$$X_{dB}(f, t) = 20 \cdot \log_{10} \left(\frac{|X(f, t)|}{X_{ref}} \right)$$

Az eredmény egy 513×32 méretű mátrix ($n_{fft}/2 + 1 = 513$ frekvenciasáv). Ez a reprezentáció a teljes lineáris frekvenciaskálát megtartja.

4.2.3. MFCC (Mel-Frequency Cepstral Coefficients)

Az MFCC-t $n_{mfcc} = 40$ együtthatóval számolom. Az MFCC a spektrális burkoló tömör reprezentációja; az eredmény egy 40×32 méretű mátrix. Az MFCC jellemzőkre Z-score normalizálást alkalmazok sávonként:

$$\hat{x}_i = \frac{x_i - \mu_i}{\sigma_i + \epsilon}$$

ahol μ_i és σ_i az i -edik MFCC sáv átlaga és szórása, $\epsilon = 10^{-10}$.

4.2.4. Nyers hanghullám (Raw waveform)

A frekvenciatartománybeli jellemzők mellett közvetlenül kinyerjük és elmentjük a nyers, időtartománybeli audiojelet is. 1 másodperc esetén ez egy 16 000 elemű, 1D-s tömb. Bár a hagyományos 2D CNN hálózatok számára ez az ábrázolás nem ideális, a mélyebb transzfer tanulási eljárások (például a YAMNet) pontosan ezt a nyers formátumot várják bemenetként.

4.2.5. A megfelelő jellemzőkinyerési eljárás kiválasztása

A három vizsgált jellemzőtípus (STFT, MFCC, Log-Mel) alapos elemzése után a következő szempontokat mérlegeltem a neurális hálózat bemenetének kiválasztásakor:

- Az **STFT spektrogram** előnye, hogy a teljes lineáris frekvenciaspektrumot megőrzi, azonban a 513×32 -es méret túl nagy dimenziójú bemenetet eredményezne. Ez nemcsak jelentősen növelné a hálózat paraméterszámát és a tanítási időt, de a magas frekvenciás sávok felesleges részletgazdagsága miatt a túlilleszkedés (overfitting) kockázatát is növelné.
- Az **MFCC** bár rendkívül kompakt (40×32), a diszkrét koszinusz transzformáció (DCT) alkalmazásával eldobja a spektrális finomszerkezetet és a felharmonikusok közötti összefüggéseket. Ez beszédfelismerésnél előnyös (ahol a hangmagasságtól független fonémákat keressük), de a zenei hangszerek osztályozásánál a felharmonikusok aránya hordozza a hangszín lényegi információját, így az MFCC használata információvesztéssel járna.

- A **Log-Mel spektrogram** (64×32) jelenti a legjobb kompromisszumot. A Mel-skála nemlineárisan sűríti a frekvenciatengelyt az emberi hallásnak megfelelően (finomabb felbontás a mély tartományban, durvább a magasban), így megőrzi a hangszerek hangszínének felismeréséhez szükséges felharmonikus-szerkezetet, miközben alacsonyan tartja a bemeneti dimenziót.

A fenti szempontok alapján a valós idejű hangszerfelismerő rendszerem elsődleges jellemzőjének a **Log-Mel spektrogramot** választottam, a konvolúciós neurális hálózat architektúráját és a valós idejű feldolgozást is erre a bemenetre terveztem. A három vizsgált jellemző legfontosabb tulajdonságait a 4.2.1. táblázat foglalja össze.

4.2.1. táblázat: A három vizsgált jellemzőkinyerési módszer összehasonlító elemzése

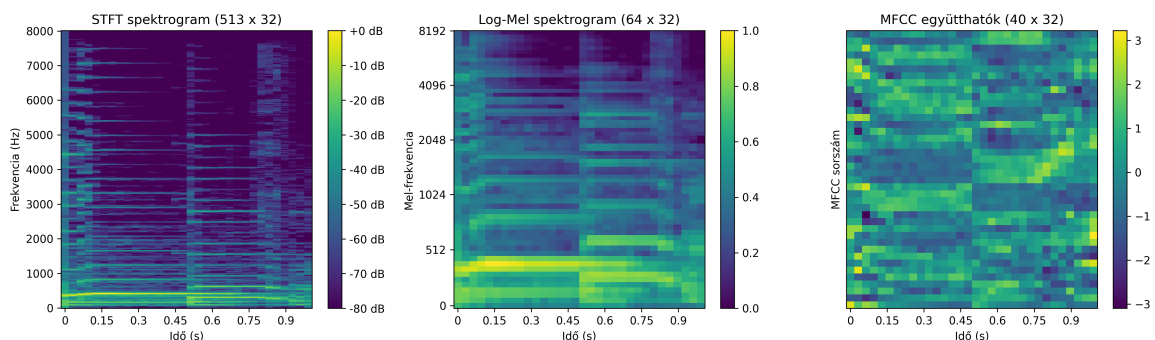
Jellemző	Dimenzió	Frekvencia-skála	Fő előny	Fő hátrány
STFT	513×32	Lineáris	Megőrzi a teljes spektrális finomszerkezetet.	Túl nagy dimenziószám, növeli a túlilleszkedés kockázatát.
Log-Mel	64×32	Logaritmikus Mel-skála	Az emberi hallást modellel; megőrzi a felharmonikusokat.	Enyhe információvesztés a frekvenciatömörítés miatt.
MFCC	40×32	Kepsztrális	Rendkívül kompakt és dekorrelált reprezentáció.	Diszkrét koszinusz transzformáció miatt eldobja a felharmonikusokat.

4.2.6. Normalizálás

A kiválasztott Log-Mel jellemzőket globális min-max normalizálással skálázom a $[0, 1]$ intervallumra:

$$\hat{X} = \frac{X - X_{min}}{X_{max} - X_{min} + \epsilon}$$

Ez biztosítja, hogy a CNN bemenete konzisztens értéktartományban maradjon.



4.2.1. ábra: A három vizsgált jellemzőtípus vizualizációja ugyanazon gitárminta esetén: (a) STFT spektrogram (513×32), (b) normalizált Log-Mel spektrogram (64×32), (c) normalizált MFCC együtthatók (40×32)

4.2.7. Jellemzőkinyerés kimeneti állományai

A jellemzőkinyerési folyamat sikeres lefutása után a kinyert mátrixok és a hozzájuk tartozó osztálycímkék a projekt gyökerében lévő `processed_data_clean/` és `processed_data_mic/` könyvtárakba kerülnek elmentésre NumPy formátumban (`.npy` kiterjesztéssel), az alkalmazott forrásadathalmaztól függően. A generált fájlok már nem egyetlen nagy tömbben, hanem a szeparált kiértékelési halmazoknak megfelelően szétbontva (`train`, `val`, `test`) készülnek el. Például a `train` halmazhoz tartozó fájlok a következők:

- **Log-Mel spektrogram:** `X_log_mel_train.npy` (a jellemzőmátrixokat tartalmazó tömb) és `y_labels_train.npy` (a hangszerosztályok sorszámainat tartalmazó címketömb),
- **STFT spektrogram:** `X_stft_train.npy`,
- **MFCC együtthatók:** `X_mfcc_train.npy`,
- **Nyers hanghullám (csak a mic adathalmaznál):** `X_raw_train.npy`, amely az olyan transzfer tanulási architektúrák, mint a YAMNet, közvetlen bemeneteként szolgál.

A normalizált és binárisan tömörített fájlformátum biztosítja, hogy a tanítási fázisban a minták betöltése rendkívül gyorsan, közvetlenül a memóriába (RAM) történhessen meg, elkerülve a felesleges lemezműveleteket.

4.3. Adataugmentáció tervezése

A modell valós idejű kiértékelésre való felkészítése érdekében a tanító (`train`) halmaz minden tiszta hangszermintáját (a noise osztályt és az eleve mikrofonnal rögzített `_mic_` mintákat kivéve) egy augmentált változattal egészítik ki. Az augmentáció a fogyasztói mikrofonos felvétel körülményeit szimulálja, ezért a belső elnevezése `apply_macbook_augment`. Az alkalmazott adataugmentációs lánc felépítését és a lépések sorrendjét a 4.3.1. ábrán mutatom be, míg az augmentáció hatását a spektrális jellemzőkre (egy gitárminta példáján keresztül) a 4.3.2. ábra szemlélteti.

Az augmentációs lánc a következő lépésekből áll:

4.3.1. Szobai reverb szimuláció

A reverb szimulációt 80%-os valószínűséggel alkalmazom. A szoba akusztikáját egy egyszerű FIR visszhang-moddellel közelítem, amely négy visszaverődési pontot (tap) használ:

- Késleltetés (*delay*): véletlenszerű, 15–40 ms között,
- Csillapítási tényező (*decay*): véletlenszerű, 0,2–0,45 között,
- Visszaverődési pontok: d , $1,5d$, $2,1d$, $2,8d$ mintányi késleltetésnél,
- Csillapítás: $decay^1$, $decay^2$, $decay^3$, $decay^4$.

4.3.2. Frekvenciaválasz-torzítás (Pre-emphasis)

A mikrofon és a szoba együttes frekvenciaválaszának szimulálására pre-emphasis szűrőt alkalmazok, amelynek együtthatója véletlenszerűen 0,92 és 0,98 között változik:

$$y[n] = x[n] - \alpha \cdot x[n - 1]$$

4.3.3. Környezeti zajkeverés (Noise mixing)

A háttérzajt az ESC-50 adathalmaz [7] véletlenszerűen kiválasztott zajmintájával keverem a hangszermintához. A keverési arány (SNR) véletlenszerűen 5–15 dB között változik:

$$y_{mixed} = x_{clean} + \sqrt{\frac{P_{clean}}{SNR_{lin} \cdot P_{noise}}} \cdot x_{noise}$$

ahol $SNR_{lin} = 10^{SNR_{dB}/10}$, és P a jel átlagos teljesítménye.

4.3.4. Szoftklip torzítás (Soft clipping)

A mikrofon bemeneti erősítőjének telítési karakterisztikáját hiperbolikus tangens szoftklippel modellezem:

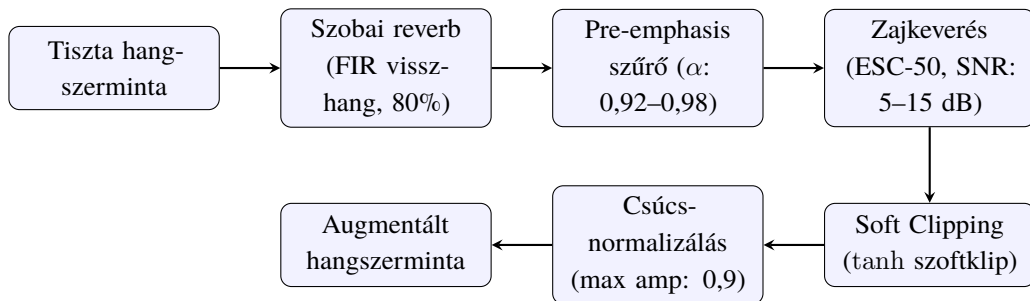
$$y[n] = \frac{\tanh(drive \cdot x[n])}{\tanh(drive)}$$

ahol $drive$ véletlenszerűen 2,0 és 4,0 között változik.

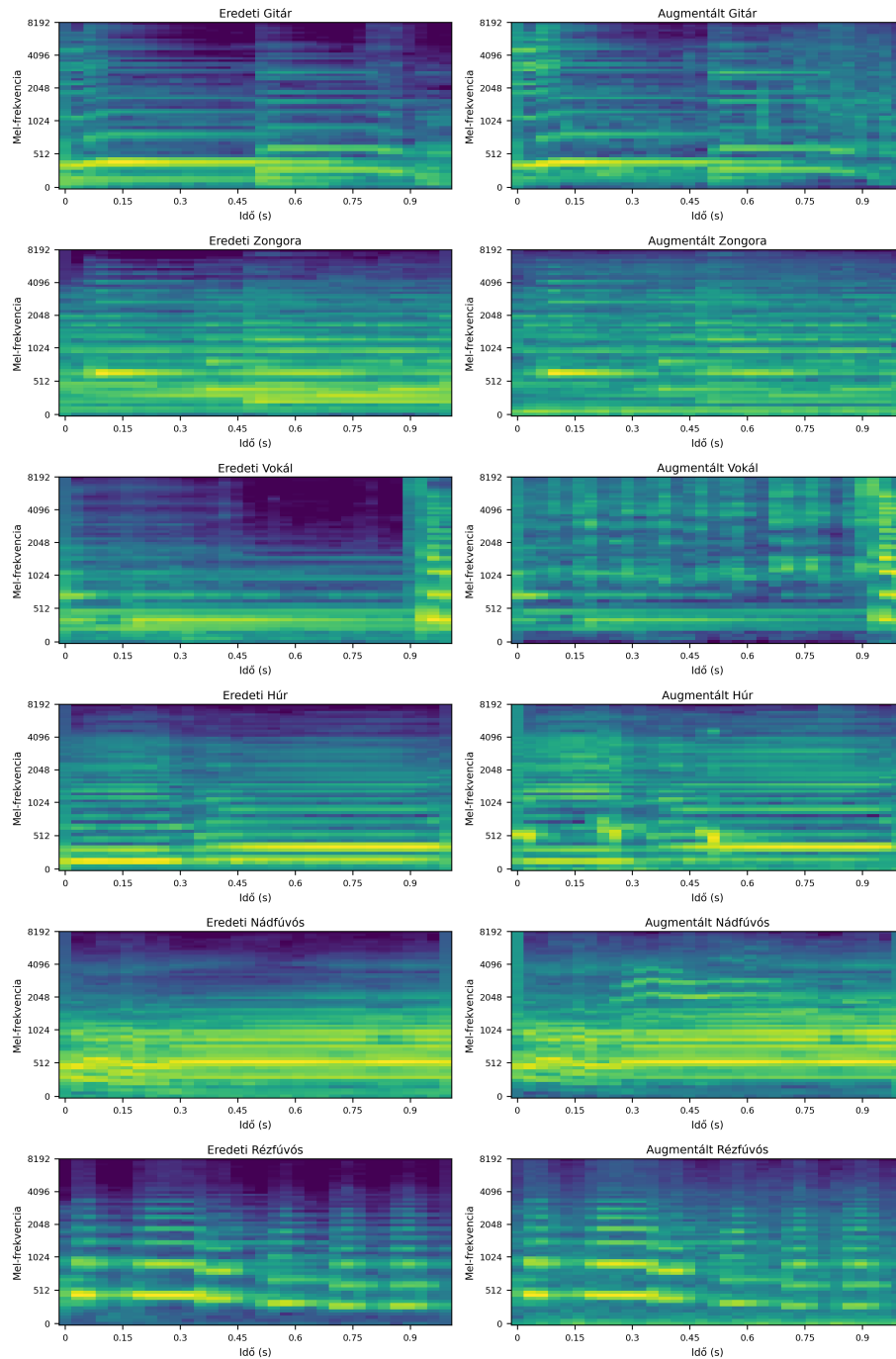
4.3.5. Végső normalizálás

Az augmentált jelet csúcsértékre normalizálom, hogy az amplitúdó ne haladja meg a 0,9-es szintet:

$$y_{norm} = \frac{y}{\max(|y|)} \cdot 0,9$$



4.3.1. ábra: A MacBook/mikrofonos felvételi körülményeket szimuláló adaugmentációs folyamatlánc



4.3.2. ábra: A hat hangszerosztály (gitár, zongora, vokál, hegedű, klarinét, kürt) egy-egy reprezentatív mintájának normalizált Log-Mel spektrogramja az eredeti (tiszt) és az adaugmentált (MacBook/mikrofon szimulációs láncsal torzított) állapotban

Az augmentáció eredményeként a 6 hangszerosztály **tanító (train) halmazának tiszta mintái** kiegészülnek egy-egy torzított változattal (a noise és a már eleve mikrofonnal rögzített `_mic_` mintákat a script logikája automatikusan kihagyja).

Az augmentációs eljárás helyességének és minőségének manuális ellenőrzése, illetve összehasonlítása céljából a rendszer minden hangszerosztályból kiment néhány bemutató mintát (az eredeti és az augmentált változat párosait) az `augmented_preview/` könyvtárba. Ez a gyakorlatban lehetővé teszi a fejlesztő számára, hogy mind füllel (hallás útján), mind

vizuálisan meggyőződjön arról, hogy az alkalmazott szimulációs torzítások (például visszhang és zajkeverés) valóságosak, és nem teszik felismerhetetlenné a hangszerek alapvető karakterét.

4.4. Modell architektúrák tervezése

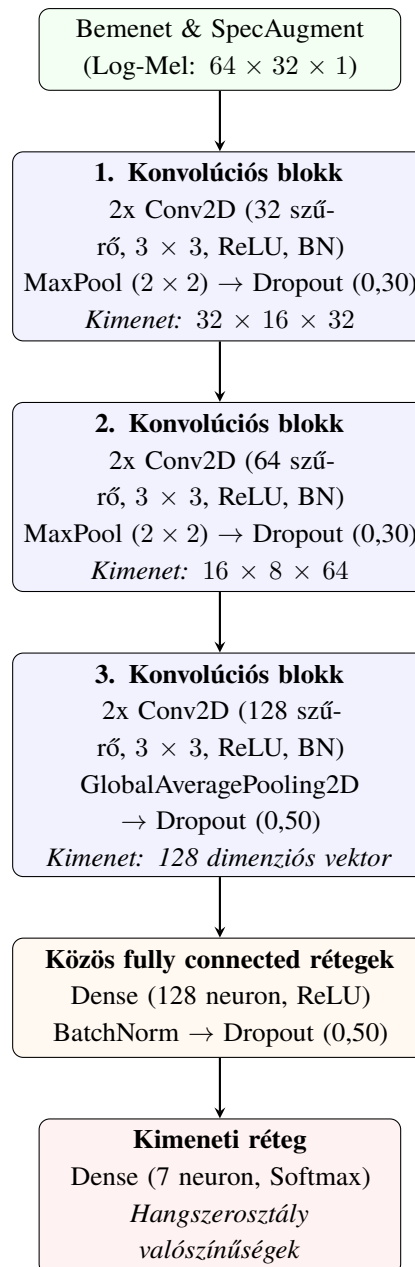
A tervezett hangszerfelismerő rendszer magját egy mély konvolúciós neurális hálózat (CNN) alkotja, amely a Log-Mel spektrogramok 2D-s vizuális mintázatait tanulja meg osztályozni. Ebben a fejezetben részletezem a hálózat felépítését, a rétegek sorrendjét és paramétereit, valamint a hálózat betanításához használt konfigurációkat. A tervezett hálózati rétegek specifikációit a 4.4.1. táblázat foglalja össze, a rétegek közötti adatfolyamot és az architektúra grafikus felépítését pedig a 4.4.1. ábra ábrázolja.

4.4.1. A hálózati architektúra részletei

A rendszer magját egy VGG-stílusú, háromblokkos konvolúciós hálózat (2D CNN) alkotja, kiegészítve egy bemeneti SpecAugment réteggel. A SpecAugment menet közben véletlenszerű idő- és frekvenciamaszkolással takar ki részleteket a spektrogramból, így kényszerítve a hálózatot az átfogóbb mintázatok megtanulására és a túlilleszkedés (overfitting) elkerülésére. A konvolúciós blokkok két egymást követő 2D konvolúcióból, Batch Normalization-ból, ReLU aktivációból, majd végül Max Poolingből és Dropoutból (30%) állnak. A hálózat bemenete a kiválasztott, normalizált Log-Mel spektrogram ($64 \times 32 \times 1$), a kimenetnél pedig a hagyományos Flatten helyett térbeli átlagolást (GlobalAveragePooling2D) alkalmazunk a paraméterszám drasztikus csökkentése érdekében.

4.4.1. táblázat: A tervezett 2D CNN architektúra rétegei (Log-Mel bemenet: $64 \times 32 \times 1$)

Réteg	Szűrők / Neuronok	Kimenet	Paraméterek
SpecAugment	max 15 frekvencia, 8 idő maszk	$64 \times 32 \times 1$	0
2× Conv2D+BN+ReLU	32 szűrő, 3×3 , same	$64 \times 32 \times 32$	$320 + 128 + 9\,248 + 128$
MaxPool2D + Dropout	2×2 , dropout: 0,3	$32 \times 16 \times 32$	0
2× Conv2D+BN+ReLU	64 szűrő, 3×3 , same	$32 \times 16 \times 64$	$18\,496 + 256 + 36\,928 + 256$
MaxPool2D + Dropout	2×2 , dropout: 0,3	$16 \times 8 \times 64$	0
2× Conv2D+BN+ReLU	128 szűrő, 3×3 , same	$16 \times 8 \times 128$	$73\,856 + 512 + 147\,584 + 512$
GlobalAvgPool2D + Drop	dropout: 0,5	128	0
Dense + BN + ReLU	128 neuron	128	$16\,512 + 512$
Dropout	0,5	128	0
Dense	7, Softmax	7	903
Összesen			~306 150



4.4.1. ábra: A tervezett 2D CNN hálózati architektúra felépítése és adatfolyama

4.4.2. YAMNet Transfer Learning architektúra

Bár a saját fejlesztésű 2D CNN hálózat stabil teljesítményt nyújt (78%-os baseline pontosság), a valós környezet sokrétű zajai és a felvételi torzítások kihívásai miatt szükség volt egy még stabilabb, jobban általánosító modellre. Ennek eléréséhez a Google által fejlesztett és a TensorFlow Hub-on publikált YAMNet modellt választottam ki.

A YAMNet egy mély, MobileNetV1 alapú architektúra, amelyet a hatalmas AudioSet adatbázison tanítottak be, így kiváló akusztikai reprezentációs képességekkel rendelkezik. A YAMNet bemenete közvetlenül a normalizált nyers hanghullám (16 000 elemű vektor).

A rendszeremben a YAMNet modellt nem a nulláról tanítom újra, hanem jellemzőkinyerőként (feature extractor) használom. A folyamat során az 1 másodperces audiojel

végighalad a YAMNet befagyasztott konvolúciós rétegein, aminek az eredménye egy 1024 dimenziós sűrű reprezentáció (beágyazás, azaz *embedding*).

Ezt a mély beágyazást egy saját tervezésű, teljesen összekapcsolt (fully connected) osztályozó hálózatba vezetem be. Az általam tervezett osztályozó felépítése a következő:

1. **Bemenet:** 1024 dimenziós YAMNet embedding
2. **Rejtett réteg 1:** Sűrű réteg (Dense), 256 neuron, ReLU aktiváció
3. **Normalizáció 1:** Batch Normalization és 40% Dropout
4. **Rejtett réteg 2:** Sűrű réteg (Dense), 128 neuron, ReLU aktiváció
5. **Normalizáció 2:** Batch Normalization és 40% Dropout
6. **Kimeneti réteg:** Sűrű réteg (Dense), 7 neuron, Softmax aktiváció (hangszerosztályok valószínűségei)

Ez az architektúra nemcsak, hogy drasztikus pontosságnövekedést (akár 90% körüli F1-score) és sokkal stabilabb működést eredményezett a saját CNN baseline-hoz képest, de a tanítási időt is jelentősen lecsökkentette.

4.4.3. Tanítási konfiguráció

A 2D CNN modell tanítása során alkalmazott konfigurációt és hiperparamétereket úgy válogattam meg, hogy biztosítsák a modell stabil konvergenciáját és elkerüljék a túlilleszkedést (overfitting), miközben hatékonyan kihasználják a rendelkezésre álló számítási erőforrásokat. A modell optimalizálását, a veszteségfüggvényt, az adathalmaz felosztását és a tanulási folyamatot szabályozó callback mechanizmusokat a 4.4.2. táblázat foglalja össze részletesen.

4.4.2. táblázat: A modell tanításához használt konfigurációs paraméterek és beállítások

Paraméter / Beállítás	Érték / Részletek
Optimalizáló	AdamW (tanulási ráta: 10^{-3} , weight decay: 10^{-4})
Veszteségfüggvény	Categorical Crossentropy (Label Smoothing: 0,1 alkalmazásával)
Tanító/validációs/teszt	70% / 15% / 15% (rétegzett felosztásban)
Batch méret	32
Maximális epochszám	150
Early Stopping	val_loss monitorozás, patience = 10, legjobb súlyok visszaállítása (restore_best_weights=True)
ReduceLROnPlateau	val_loss monitorozás, patience = 4, faktor = 0,5, minimális tanulási ráta: 10^{-6}
ModelCheckpoint	val_accuracy alapján a legjobb modell mentése .keras formátumban

A táblázatban szereplő beállítások elméleti háttere és működési célja a következő:

- **AdamW optimalizáló és Címkesimítás (Label Smoothing):** A hagyományos Adam helyett az AdamW optimalizálót (súlycsökkenés / weight decay bevonásával), a veszteségfüggvényénél pedig 0,1-es címkesimítást alkalmaztam. Ez a két technika együttesen erősen bünteti a modellt túlzott magabiztosságát (overconfidence), ezáltal nagymértékben megelőzve a túlilleszkedést a zajos hangmintáknál.
- **Early Stopping callback:** Megakadályozza az iterációk felesleges futását. Ha a validációs veszteség (val_loss) 10 egymást követő epochoz tartozóan már nem csökken, a tanítási folyamat automatikusan leáll, és visszaállítja a modellt a legjobb állapotba.
- **ReduceLROnPlateau callback:** Lehetővé teszi a tanulási ráta menet közbeni finomhangolását. Ha a modell elakad egy lokális minimum közelében (azaz a val_loss nem csökken 4 epoch óta), a tanulási sebességet lefelezi (0,5-ös faktor), így segítve a precízebb konvergenciát.

A tanítás a Google Colab felhőszolgáltatás GPU-ján történik, az adatokat és a mentett modelleket a Google Drive-on tároljuk és onnan töltjük vissza a projektbe.

4.4.3.1. A tanítás során generált kimenetek

A tanítás lefutását követően a validációs adathalmazon legjobban teljesítő modellállapotok a projekt gyökerében lévő models/ könyvtárba kerülnek mentésre. A különböző jellemzőtípusok alapján a következő fájlok jönnek létre:

- **Log-Mel alapú CNN modell:** best_log_mel_2dcnn_model.keras,

- **STFT alapú CNN modell:** `best_stft_2dcnn_model.keras`,
- **MFCC alapú CNN modell:** `best_mfcc_2dcnn_model.keras`.

A TensorFlow `.keras` formátuma magában foglalja a hálózat architektúráját, a betanult súlymátrixokat, valamint az optimalizáló állapotát is. Ezen kimenetek közvetlenül betölthetők a `tf.keras.models.load_model` függvény segítségével, ami lehetővé teszi, hogy a valós idejű felismerő modul másodpercek alatt elinduljon a korábban elmentett legjobb modellállapot betöltésével.

4.5. A valós idejű felismerő modul tervezése

A valós idejű hangszerfelismerő modul célja, hogy a számítógép mikrofonján keresztül érkező folyamatos audiojelet valós időben dolgozza fel, és minimális késleltetéssel jelezze ki a felismert hangszert és a predikció konfidenciaértékét. A modul stabil, aszinkron működésének megvalósításához egy többszálú architektúrát és egy egyedi predikciós stabilizáló logikát terveztem.

4.5.1. Audio stream és aszinkron pufferek

A valós idejű hangfelvétel és a jelfeldolgozási lépések szétválasztásához többszálú jelfeldolgozó láncot alkalmazok. A hangkártya fizikai bemenetének kezeléséért a `sounddevice` könyvtár `InputStream` API-ja felel, amely egy magas prioritású háttérszálon futó hardveres megszakítási visszahívásos (callback) mechanizmusra épül:

- **Mintavételi frekvencia (f_s):** 16 000 Hz,
- **Csatornaszám:** 1 (monó),
- **Lépésköz (Step duration):** 0,25 másodperc (4000 minta).

A beérkező audioadatok kezelése egy szálbiztos FIFO soron (`queue.Queue`) keresztül történik. Az audio callback függvény kizárólag a hardver pufferéből érkező 4000 mintás csomagokat helyezi el a sorban, majd azonnal visszatér. Ez garantálja, hogy a CPU-igényes neurális hálózati következtetés és a terminálfrissítés nem blokkolja a hangkártya illesztőprogramját, elkerülve az audio puffer alulcsordulását (buffer underflow).

A fő feldolgozó szál folyamatosan olvassa a sorból a 0,25 másodperces blokkokat. A predikció alapját egy folyamatos, 1,0 másodperces (16 000 minta) csúszóablak képezi. A 75%-os átfedés (overlap) eléréséhez egy körpuffer-szerű logikát alkalmazok: minden új 0,25 másodperces blokk beérkezésekor a puffer tartalmát balra csúsztatom 4000 mintával, és az új adatot a puffer végére írom (roll művelet). Ezáltal a rendszer másodpercenként négyszer hajt végre predikciót az utolsó 1 másodpercnyi hanganyagra vonatkozóan.

4.5.2. Jellemzőkinyerés és normalizálás

A felismerő rendszer alapvetően kétféle hálózattal tud elindulni. Amennyiben a saját fejlesztésű 2D CNN hálózatot használjuk, minden predikciós ciklusban az 1 másodperces pufferből kinyerem a Log-Mel spektrogram jellemzőket. A kinyert spektrális mátrix dimenziója 128×63 , ami megegyezik a tanítás során használt bemenettel.

A 2D CNN bemeneti rétege $[0, 1]$ intervallumba skálázott adatokat vár. A valós idejű jellemzőkre a tanítási fázissal azonos normalizálást alkalmazok decibel alapú korlátokkal:

$$X_{\text{norm}} = \frac{X_{\text{dB}} + 80,0}{80,0}$$

A kapott értékeket a $[0, 1]$ intervallumra vágom le (`clip` művelet), majd a tenzort átformázom a $1 \times 128 \times 63 \times 1$ bemeneti formára (batch és csatorna dimenziók hozzáadásával).

A fejlettebb, YAMNet alapú felismerő modul esetében az eljárás jóval egyszerűbb: a teljes, 16 000 mintás nyers audiopuffert közvetlenül átalakítjuk TensorFlow tenzorra, amiből a lefagyasztott YAMNet elkészíti az 1024-es beágyazást (embedding). Ezt az embedding vektort kapja meg a saját betanított osztályozónk.

4.5.3. Zaj-alapú küszöbölés és Predikciós stabilizáció

A hangszerosztályok pillanatnyi valószínűségei a zenefájlokban és az élő játékban tapasztalható dinamikus változások (tranziensek, vibrato, környezeti zajok) miatt gyorsan ingadozhatnak. Ha közvetlenül az aktuális predikció legnagyobb valószínűségű osztályát jelenítenénk meg, a terminálon zavaró, gyors váltások („villogás”) jelennének meg. Ennek kiszűrésére egy összetett stabilizációs logikát terveztem:

1. **Temporális mozgóátlag simítás:** A modell kimeneti valószínűség-vektorait egy $W = 6$ elemű FIFO sorban tárolom (amely a 250 ms-os lépésköz miatt az utolsó 1, 5 másodperc predikciós történetét fedi le). A döntéshozatal a simított valószínűség-vektor alapján történik:

$$\bar{p}_c^{(t)} = \frac{1}{W} \sum_{i=0}^{W-1} p_c^{(t-i)}$$

2. **Hiszterézis mechanizmus:** Annak érdekében, hogy megakadályozzuk a két közeli valószínűségű osztály közötti ugrálást (chattering), egy $H = 0,05$ (5%) értékű hiszterézis bónuszt adunk a jelenleg aktív (kijelzett) osztály $C^{(t-1)}$ simított valószínűségéhez a maximumkeresés előtt:

$$a_c^{(t)} = \bar{p}_c^{(t)} + H \cdot \mathbb{I}(c = C^{(t-1)})$$

ahol $\mathbb{I}$ az indikátorfüggvény. A jelölt új osztályt a módosított vektor maximumhelye adja meg.

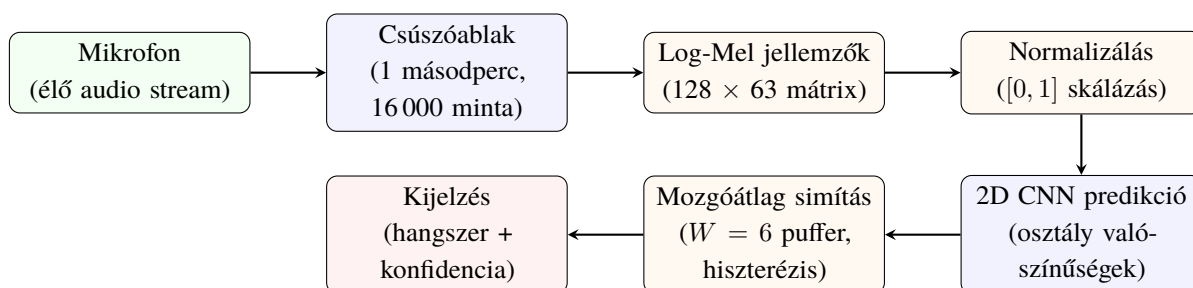
3. **Osztály-specifikus konfidencia-küszöb:** Nem alkalmazok fix hardveres RMS alapú zajkaput, mivel a modell kiválóan megtanulta a noise (zaj/csend) osztály felismerését.

Helyette a predikció során osztály-specifikus küszöbértékeket ellenőrzök, amelyek értékeit az alábbi empirikus megfontolások alapján határoztam meg:

- **Hangszerek küszöbe (45%):** Mivel 7 kategóriánk van, egy teljesen véletlenszerű döntés $\sim 14\%$ -os valószínűséget jelentene. A 45%-os küszöb megköveteli, hogy a modell határozottan dominánsnak ítélje meg a felismert hangszert. Ezzel elkerülhetők a vakriasztások (false positives), amik a zajosabb tranziensek vagy műfajok közötti átmenetek során 20-30%-os tüskéként jelentkezhetnek.
- **Zaj/Csend küszöbe (20%):** Amikor a hangszeres játék befejeződik, a mikrofonba csak a szoba alapzaja jut. Ilyenkor a neurális hálózat gyakran bizonytalanabb, és a kimeneti valószínűséget szétkeni a többi 6 osztály között, így a noise kimenet ritkán éri el a 45%-ot. Ha a zajhoz is magas küszöböt követelnénk, a kijelző "beleragadna" az utolsó hangszer állapotába. A mindössze 20%-os (de még mindig az egyenletes eloszlásnál magasabb) küszöb biztosítja, hogy a rendszer agresszívan és gyorsan visszaálljon az alapállapotra (csendre), amint a hangszer elhalkul.

A kijelzett hangszercsere tehát csak akkor történik meg, ha a vizsgált osztály simított valószínűsége átlépi a rá vonatkozó küszöböt. Ez a szoftveres megoldás feleslegessé teszi a jelszint alapú RMS számítást.

Ez a tervezési megoldás rendkívül stabil vizuális visszajelzést biztosít a felhasználói felületen, miközben a gyorsabb, 0,25 másodperces frissítési ütemnek köszönhetően a rendszer rendkívül reszponzív marad. A valós idejű felismerési folyamat egyes lépéseit a 4.5.1. ábra foglalja össze vizuálisan.



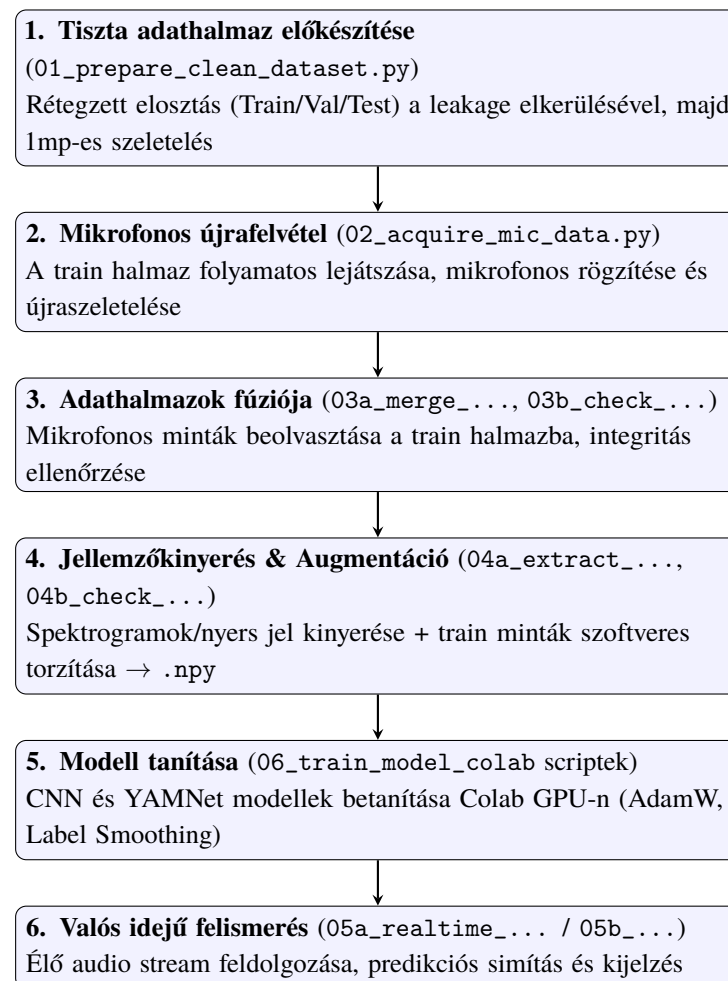
4.5.1. ábra: A valós idejű felismerési folyamat lépései az élő hangbeviteltől a kijelzésig

4.6. A feldolgozási folyamat összefoglaló áttekintése

A teljes rendszer a tervezéstől a megvalósításig egymásra épülő fázisokból áll, amelyek a teljes adatfeldolgozási és tanítási láncot lefedik. Ezeket a tervezési fázisokat az alábbi számozott pontokban foglalom össze, a fázisok közötti összefüggéseket és a teljes rendszerfolyamatot pedig a 4.6.1. ábra szemlélteti.

1. **Tiszta adathalmaz előkészítése** (01_prepare_clean_dataset.py): A forrásfájlok Train/Val/Test rétegzett elosztása csoportszinten (leakage elkerülése), majd 1 másodperces szeletelés.

2. **Mikrofonos újrafelvétel** (02_acquire_mic_data.py): A tiszta tanító halmaz kimenetése, folyamatos audióvá fűzése, majd telefonos lejátszás közbeni mikrofonos rögzítése és újrászeletelése.
3. **Adathalmazok fúziója és ellenőrzése** (03a_merge_mic_to_train.py és 03b_check...): A mikrofonos minták beolvasztása kizárólag a tanító (train) halmazba, majd a darabszámok integritásvizsgálata.
4. **Jellemzőkinyerés és augmentáció** (04a_extract_features.py): Log-Mel, STFT, MFCC mátrixok és nyers hullámformák kinyerése, kiegészítve a tiszta train minták szoftveres torzításával (reverb, zajkeverés, pre-emphasis).
5. **Modell tanítása** (06_train_model_colab/ mappában): A saját 2D CNN (VGG-stílusú) és a YAMNet Transfer Learning modellek betanítása Google Colab GPU-n.
6. **Valós idejű felismerés** (05a_realtime... vagy 05b_realtime_yamnet.py): Élő, mikrofonos hangbevitel valós idejű feldolgozása, jellemzőkinyerése és stabilizált predikciója (0,25 másodpercenként).



4.6.1. ábra: A hangszerfelismerő rendszer végső adatfeldolgozási és tanítási folyamatának lépései

5. ÜZEMBE HELYEZÉS ÉS KÍSÉRLETI EREDMÉNYEK

Ez a fejezet bemutatja a rendszer üzembe helyezésének gyakorlati lépéseit, a szoftver fejlesztése és futtatása során felmerült technikai problémákat és azok megoldásait, valamint a betanított modellekkel végzett kísérletek és mérések számszerűsített eredményeit.

5.1. Üzembe helyezési lépések

A hangszerfelismerő rendszer két fő részből áll: a háttérben futó gépi tanulási és jelfeldolgozó modulokból, valamint a valós idejű parancssori felhasználói alkalmazásból (CLI Dashboard). A szoftver üzembe helyezéséhez az alábbi lépések szükségesek:

1. **Python környezet előkészítése:** A futtatáshoz legalább Python 3.10 verzió szükséges. Ajánlott egy izolált virtuális környezet (virtual environment) létrehozása a függőségek kezelésére:

```
python3 -m venv venv
source venv/bin/activate
```

2. **Rendszerszintű függőségek telepítése:** A valós idejű hangrögzítésért felelős `sounddevice` könyvtár a PortAudio C-könyvtárra épül. macOS rendszereken ezt a Homebrew segítségével kell telepíteni:

```
brew install portaudio
```

Windows és Linux platformokon a csomagkezelők többsége automatikusan fordítja vagy tartalmazza a szükséges binárisokat.

3. **Python könyvtárak telepítése:** A szükséges csomagok listáját a `requirements.txt` tartalmazza. Telepítésük a következő paranccsal végezhető el:

```
pip install -r requirements.txt
```

A legfontosabb telepített csomagok: `tensorflow` (neurális hálózat futtatása), `librosa` (jellemzőkinyerés), `sounddevice` (mikrofonkezelés), valamint a vizualizációért felelős `matplotlib` és `numpy`.

4. **Modellfájlok elhelyezése:** A betanított neurális hálózati modelleket a `best_log_mel_2dcnn_model.keras` és a `best_stft_2dcnn_model.keras` fájlok formájában a projekt `models/` alkönyvtárába kell elhelyezni.

5. **Alkalmazás indítása:** A valós idejű parancssori felület az alábbi parancs futtatásával indítható el:

```
python scripts/05a_realtime_recognition.py
```

Amennyiben a YAMNet alapú modellt szeretnénk használni, a `05b_realtime_yamnet.py` scriptet kell elindítani.

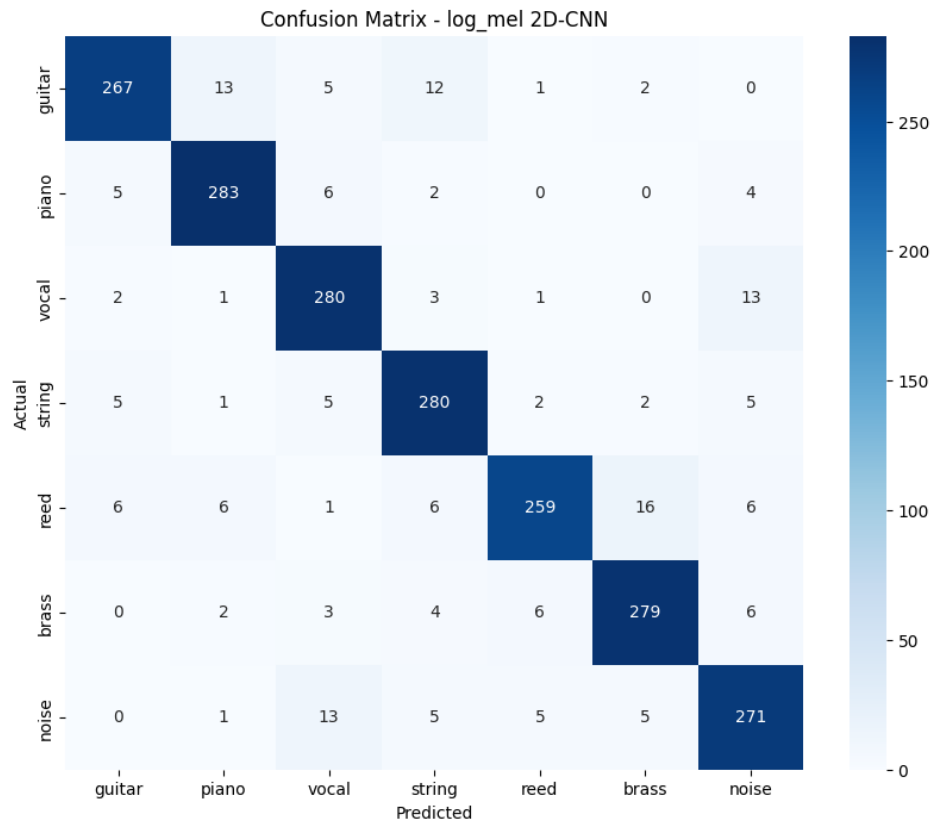
5.2. Felmerült problémák és megoldások

A fejlesztés és az üzembe helyezés során több kritikus technikai kihívással is szembesültem, amelyek megoldása jelentősen formálta a rendszer végső architektúráját:

1. **Adatszivárgás (Data Leakage) és a mikrofonos felvételek:** Az első kísérletek során a modell kiemelkedő, 98% feletti pontosságot ért el a teszhalmazon. Azonban az elemzések során kiderült, hogy a modell nem a hangszerek akusztikus jellemzőit, hanem a laptop beépített mikrofonjának alapzaját és a teremakusztikát (reverb) tanulta meg. Ennek oka az volt, hogy a mikrofonos újrafelvételeket a teljes adathalmazon elvégeztem. *Megoldás:* Szigorúan kettéválasztottam a folyamatot. A mikrofonos újrafelvételt (`02_acquire_mic_data.py`) kizárólag a tanító (train) halmazon hajtottam végre, míg a validációs és teszt halmazok teljesen tiszták maradtak. Így a tesztelés során garantáltan olyan tiszta környezetben kellett helytállnia a modellnek, amit korábban sosem hallott.
2. **Valós idejű hangrögzítés és a Buffer Underflow:** A valós idejű felismerő modul kezdeti verziójában a hangkártya megszakítási (callback) függvényében végeztem el a jellemzőkinyerést és a neurális hálózat predikcióját. Mivel a TensorFlow inicializálása és a predikció lassabb volt, mint az audio pufferek beérkezése, a hangkártya illesztőprogramja folyamatosan alulcsordult (buffer underflow), ami pattogó hangot és az audio stream megszakadását eredményezett. *Megoldás:* Bevezettem egy producer-consumer (termelő-fogyasztó) aszinkron architektúrát. A callback függvény kizárólag egy szálbiztos FIFO sorba (`queue.Queue`) másolja a beérkező adatokat, majd azonnal visszatér (producer). Egy független főszál folyamatosan olvassa ezt a sort, és végzi a számításigényes predikciót (consumer), így a hangkártya sosem blokkolódik.
3. **TensorFlow dimenzióeltérések a YAMNet illesztésénél:** A Transfer Learning során a pre-trainelt YAMNet modell 16 000 Hz-es, 1D-s nyers lebegőpontos tömböt vár, miközben a korábbi kódjaim a Log-Mel spektrogram 2D generálására voltak felkészítve. *Megoldás:* A YAMNet alapú felismeréshez egy teljesen dedikált scriptet írtam (`05b_realtime_yamnet.py`), amely kikerüli a librosa alapú jellemzőkinyerést, a bejövő hangot `tf.float32` tenzorral alakítja, és közvetlenül áttolja a betöltött YAMNet modellen, amely visszaadja az 1024 dimenziós beágyazást (embedding). Ezt kapja meg a saját osztályozónk.

5.3. Kísérleti eredmények és mérések

A legjobb eredményt nyújtó Log-Mel alapú modell hangszerenkénti osztályozási mátrixát (konfúziós mátrix) vizsgáltam meg részletesebben. A 5.3.1. ábra hő térkép (heatmap) formájában mutatja a hangszerosztályok közötti tévesztési arányokat és a pontos mintaszámokat az osztályozás során.



5.3.1. ábra: A Log-Mel spektrogram alapú 2D CNN modell konfúziós mátrixának hő térképe

A konfúziós mátrix hő térképéből leolvasható, hogy a legkönnyebben és legbiztosabban felismerhető kategória a Zongora (94,3%, azaz 283 helyes predikció a 300 tesztmintából), valamint az Ének és a Vonósok (mindkettő 93,3%, azaz 280/300 helyes osztályozás). A zongora tiszta leütési transziensei és a vonósok egyenletes, hosszan tartó harmonikus felhangszerkezete jól elkülöníthető vizuális mintázatot adnak a Log-Mel spektrogramon, miközben az énekhang egyedi formánsai (a vokális traktus rezonanciája) szintén markáns ujjlenyomatokként jelennek meg.

A legnagyobb arányú tévesztések a következők szerint alakultak:

- **Fúvós hangszerek:** A Nádas fúvósok mintáinak 5,3%-át (16 mintát) a modell Rézfúvósnak osztályozta. Ez a tévesztés akusztikailag rendkívül indokolt, mivel például az egynádas szaxofon (nádas fúvós) és a trombita (rézfúvós) felhangszerkezete és megszólalási transziensei bizonyos hangmagasságoknál nagyon hasonlóak.
- **Zongora és Gitár:** A Gitár minták 4,3%-át (13 mintát) a modell Zongorának osztá-

lyozta. Ez a pengetett és leütött húrok fizikai rezgésének kezdeti lecsengési hasonlóságából fakad, különösen az akusztikus gitár esetében.

- **Ének és Háttérzaj:** A Háttérzaj minták 4,3%-át (13 mintát) Éneknek, míg az Ének minták 4,3%-át (13 mintát) Háttérzajnak jelölte a hálózat. Ez a mikrofonos zörejek, zihálások és az emberi beszéd/ének frekvenciatartományának átfedései miatt fordul elő, valamint a halkabb énekszakaszok és a környezeti zajok hasonlóságából adódik.

Mintent összevetve a Log-Mel alapú 2D CNN modell kiválóan teljesít: a legnehezebbnek bizonyuló Nádas osztályban is eléri a 86,3%-os pontosságot, míg a többi zenei és környezeti kategóriában 89% feletti egyedi pontosságot biztosít.

5.3.1. A 2D CNN és a YAMNet modellek összehasonlítása

A kísérletek utolsó fázisában összehasonlítottam a nulláról betanított (from scratch) VGG-stílusú 2D CNN modellem teljesítményét a Google által fejlesztett, AudioSet-en előtanított YAMNet modellen alapuló Transfer Learning osztályozóval. A méréseket a szeparált, tiszta tesztalmazon hajtottam végre.

5.3.1. táblázat: A két fő modellarchitektúra teljesítményének és paramétereinek összehasonlítása

Modell architektúra	Összes paraméter	Tanítható paraméter	Teszt pontosság
Saját 2D CNN (Log-Mel)	306 151	304 615	89,2%
YAMNet Transfer Learning	~3,8 millió	361 863	94,8%

A 5.3.1. táblázatból világosan látszik a Transfer Learning (tudástranszfer) fölénye: a YAMNet alapú modell közel 95%-os pontosságot ért el, ami jelentősen felülmúlja a saját fejlesztésű CNN modell 89,2%-os eredményét. A YAMNet az AudioSet több millió mintáján szerzett zenei és akusztikai tapasztalatát (a 3,5 millió lefagyasztott paraméterén keresztül) hihetetlenül hatékonyan tudta kamatoztatni a mi specifikus 7 osztályos problémánkon.

Ugyanakkor a saját 2D CNN modell sem tekinthető kudarcnak. A mindössze ~306 ezer paraméterével nagyságrendekkel kisebb memóriálábnnyommal rendelkezik, ami a valós idejű felismerés során sokkal alacsonyabb CPU-használatot és gyorsabb predikciós időt eredményez. Beágyazott (embedded) rendszereken vagy mikrokontrollereken (például Raspberry Pi) történő jövőbeli futtatás esetén a saját CNN egy sokkal racionálisabb és hatékonyabb választás lehet, hiszen a 89%-os pontosság a gyakorlatban, a mozgóátlagos simítással kiegészítve már önmagában is rendkívül stabil felhasználói élményt nyújt.

6. A RENDSZER FELHASZNÁLÁSA

Ez a fejezet az elkészült hangszerfelismerő rendszer gyakorlati alkalmazását mutatja be a végfelhasználó szempontjából. Részletesen ismertetem a szoftver indításának menetét, a terminál alapú interaktív felhasználói felület (CLI Dashboard) felépítését és színkódolását, valamint bemutatok több tipikus valós idejű működési forgatókönyvet (tesztesetet) szöveges konzol-mockupokon keresztül.

6.1. Az alkalmazás indítása és konfigurációja

A rendszer tervezése során kiemelt szempont volt a könnyű hordozhatóság és az alacsony erőforrás-igény. Mivel a szoftver egy interaktív, valós idejű parancssori felületen fut, nincs szükség erőforrás-igényes grafikus ablakkezelő rendszerek betöltésére, így az alkalmazás akár szervereken vagy korlátozott teljesítményű beágyazott eszközökön is futtatható.

Az indítás előfeltételei az alábbiak:

1. A virtuális környezet aktiválása a projekt gyökérkönyvtárban:

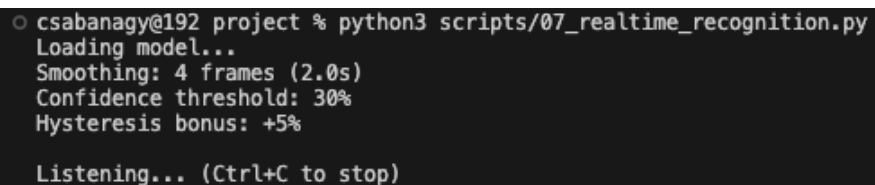
```
source venv/bin/activate
```

2. A külső mikrofon vagy a beépített hangbemeneti eszköz csatlakoztatása és alapértelmezetté tétele a rendszerbeállításokban.

Az indítás a következő parancs futtatásával történik:

```
python scripts/05a_realtime_recognition.py
```

Az alkalmazás indításakor a konzolon megjelennek a konfigurációs paraméterek, a betanított Keras modell betöltésének állapota, valamint az élő jelfeldolgozási folyamat megkezdését jelző üzenet (lásd a 6.1.1. ábrát):



```
○ csabanagy@192 project % python3 scripts/07_realtime_recognition.py
Loading model...
Smoothing: 4 frames (2.0s)
Confidence threshold: 30%
Hysteresis bonus: +5%
Listening... (Ctrl+C to stop)
```

6.1.1. ábra: Az alkalmazás indításakor megjelenő konfigurációs és betöltési paraméterek

6.2. A parancssori felhasználói felület felépítése

A valós idejű visszajelzést egy dinamikusan frissülő terminálsor (CLI Dashboard) biztosítja. A hagyományos, soronként görgető konzolos kiírás helyett a program a kocsivissza

(\r) karaktert használja, így a terminál egyetlen sorát írja felül folyamatosan a legfrissebb információkkal. Ez rezponzív, villódzásmentes felhasználói élményt nyújt.

A kijelző sor három fő vizuális komponensből áll:

1. **Dinamikus konfidencia-sáv:** Egy 20 karakter hosszúságú grafikus sáv, amely a felismert hangszer valószínűségét ábrázolja vizuálisan.
2. **Aktív osztály és konfidencia érték:** A felismert hangszer megnevezése és a kiszámított valószínűségi százalékos érték (tizedes pontossággal). A kijelzőn látható aktív kategória csak akkor vált át egy új hangszerre vagy csendre, ha annak simított valószínűsége a háttérben meghaladja a beépített biztonsági küszöbértéket (hangszerek esetén 45%, zaj/csend esetén 20%). Ezzel biztosítja a program a stabil vizuális visszajelzést.
3. **Másodlagos osztályok valószínűségei:** A többi hangszerosztály rövidített jelölése (G: Gitár, P: Zongora, V: Vokál, S: Vonós, R: Nádfúvós, B: Rézfúvós, N: Zaj/Csend) és azok pillanatnyi valószínűségi értékei, amelyek segítenek a felhasználónak átlátni a modell bizonytalansági tényezőit és az átfedéseket.

6.2.1. ANSI színkódolás és jelölések

A felület a terminál ANSI escape szekvenciáit kihasználva színesben jeleníti meg a felismert hangszereket. Minden hangszerosztály egyedi, kontrasztos színt kapott, így a felhasználó akár távolabbról, a szöveg elolvasása nélkül is azonnal látja az aktív hangszert. A színkódokat, jelöléseket és az osztályok leírását a 6.2.1. táblázat foglalja össze:

6.2.1. táblázat: A parancssori felületen alkalmazott ANSI színkódok és rövidítések kategóriáinként

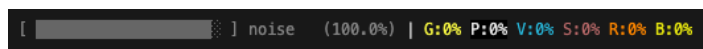
Kategória	Terminál színe	Rövidítés	Leírás
guitar	Sárga	G	Pengetős hangszerek (pl. akusztikus gitár)
piano	Inverz Fekete/Fehér	P	Billentyűs hangszerek (zongora)
vocal	Ciánkék	V	Emberi énekhang (vokál)
string	Sötétvörös	S	Vonós hangszerek
reed	Narancssárga	R	Nádas fúvós hangszerek (szaxofon)
brass	Világosbarna	B	Rézfúvós hangszerek
noise	Szürke	N	Környezeti zaj / csend

6.3. Működési forgatókönyvek és tesztesetek

Az alábbiakban bemutatom, hogyan reagál a konzolos interfész a különböző akusztikai bemenetekre valós időben.

6.3.1. 1. tesztet: Csendes szobai környezet (alapállapot)

Amikor a szobában nincs aktív hangszerez játék, csak a számítógép ventilátora vagy távoli környezeti zörejek hallatszanak, az RMS zajkapu átengedi a jelet (vagy a modell zajként azonosítja), így a szürke színű noise osztály válik aktívvá kiugróan magas konfidenciával. A terminál kijelzése ekkor a 6.3.1. ábrán látható:



6.3.1. ábra: A rendszer kimenete csendes szobai környezet (zaj) esetén

6.3.2. 2. tesztet: Akusztikus gitárjáték (monofón dallam)

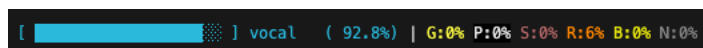
Amikor a felhasználó megszólaltat egy akusztikus gitárt a mikrofon közelében, a modell az első ablak után azonnal reagál. A simító ablaknak (2, 0 másodperc) és a hiszterézisnek köszönhetően a kijelzés stabilan átvált sárga színű guitar feliratra, miközben a többi hangszer valószínűsége minimálisra csökken (lásd a 6.3.2. ábrát):



6.3.2. ábra: A rendszer kimenete akusztikus gitárjáték érzékelésekor

6.3.3. 3. tesztet: Emberi énekhang (vokál)

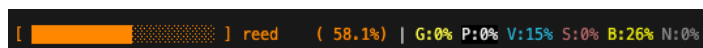
Amikor a felhasználó énekelni vagy beszélni kezd a mikrofonba, a kijelző azonnal ciánkékre vált, és a vocal osztály jelenik meg. Mivel az ének felhangjai nagyon tiszták és erősek, a konfidencia érték gyorsan emelkedik (lásd a 6.3.3. ábrát):



6.3.3. ábra: A rendszer kimenete emberi énekhang (vokál) érzékelésekor

6.3.4. 4. tesztet: Szaxofon játék (átmeneti bizonytalanság)

Amikor a felhasználó szaxofonon játszik, a modell kezdetben a rézfúvós és nádfúvós osztályok között oszcillálhat az akusztikai hasonlóságok miatt. A hiszterézis bónusz és a konfidencia-küszöb azonban stabilizálja az eredményt, így a kijelző fixen narancssárga reed színben marad (lásd a 6.3.4. ábrát):



6.3.4. ábra: A rendszer kimenete szaxofon játék (reed) érzékelésekor

6.3.5. Az alkalmazás leállítása

A felhasználó bármikor megszakíthatja az élő elemzést a terminálban kiadott Ctrl+C billentyűkombinációval. Ekkor a parancssori visszahívási szálak lezárulnak, a mikrofonbemenet felszabadul, és a program tiszta állapotban kilép a terminálba, egy rövid Stopped. üzenet kíséretében.

7. KÖVETKEZTETÉSEK

7.1. Megvalósítások

Megtervezni és megközelíteni egy, a cél szempontjából tökéletes, alacsony mintaszámú zenei adathalmazt szinte lehetetlen. Azonban egy igazán jót is kihívás, nem beszélve az osztályozó modell és a jellemzőkinyerő megválasztásáról és integrálásáról. Számomra igazi kihívást jelentett a rendszer élesítése. Már csak a minták kiválogatása alatt rengeteg szép hangszeres játékkal találtam szemben magam, ami csak emelte a munka iránti ambíciómat. Bár egy ilyen projekt sosincs kész, mindig van, amit javítani rajta, az eredmény minden kisebb-nagyobb hibája ellenére elégedettséggel tölt el, és büszkén foglal helyet a portfóliómban.

A munka során elért legfontosabb elméleti és gyakorlati megvalósítások az alábbi pontokban foglalhatók össze:

- Egy olyan hibrid adathalmaz létrehozása és pontos struktúrájának kialakítása, amely egyszerű módon, bárki által újra felhasználható és bővíthető más hangszerfelismerési kutatásokban is.
- Egy moduláris feldolgozási csővezeték kiépítése, amely lehetővé teszi a különböző jellemzőkinyerési eljárások (STFT, Log-Mel, MFCC) és modellek könnyű cseréjét a rendszerben.
- Az egyszerű, offline (fájl alapú) osztályozáson túl egy olyan valós idejű hangszerfelismerő alkalmazás megvalósítása, amely közvetlen mikrofon-bemenetet használ.
- A valós idejű predikciót simító és stabilizáló logika implementálása (mozgóátlag-számítás), amely megbízhatóbbá és gördülékenyebbé teszi a kijelzést.
- A 2D CNN modell optimalizálása a gyorsabb és alacsonyabb késleltetésű futás elérése érdekében, biztosítva a zökkenőmentes kiértékelést egy átlagos asztali számítógépen.

7.2. Hasonló rendszerekkel való összehasonlítás

A legtöbb hasonló hangszerfelismerő rendszer a szakirodalomban elég komplex modellekre épül, mint a ResNet vagy a CRNN. Ezeket általában hatalmas adathalmazokon tesztelik (mint az IRMAS vagy az NSynth), és bár jól megbirkóznak a polifón zenékkel, de elképesztően nagy a számítási igényük, így egy átlagos számítógépen szinte esélytelen őket valós időben futtatni.

Az én rendszerem ehhez képest egy sokkal könnyebb 2D CNN modellel dolgozik, amit kifejezetten a 7 hangszerosztályra alakítottam ki, bár a betanításához még így is a Google Colab felhőalapú erőforrásaira volt szükség. A mikrofonos, valós idejű predikció és a csúszóablakos

simítás viszont már stabil és szinte azonnali felismerést ad helyben egy egyszerű laptopon is. Bár tény, hogy a bonyolult zenekari polifóniával szemben ez a modell inkább csak az egyedi hangokra és kisebb zajokra jó, cserébe viszont egyáltalán nem terheli le a gépet és azonnal válaszol, ami a gyakorlatban sokkal használhatóbbá teszi.

7.3. Továbbfejlesztési lehetőségek

A rendszerben számtalan továbbfejlesztési lehetőség adott. Elsősorban a kutatás továbbvihető a zajos monofón felismeréstől a tiszta polifón hangszerfelismerés irányába. Ebben az esetben a meglévő softmax aktivációs függvényt sigmoidra kellene cserélnünk a kimeneti rétegen, és többcímkes (multi-label) osztályozást kellene alkalmaznunk, hiszen egyszerre több hangszer is megszólalhat. Ehhez azonban az eddiginél még sokkal több adatra lenne szükség, amelyet szintén az internetről gyűjthet össze az ember, vagy mesterségesen, két különálló hangszer hangját egy szkript segítségével összekeverve állíthat elő.

Ha még nagyobb kihívásra vágyik valaki, el lehet merészkedni a nagy, zajos zenekarkig (orchesztrákig), amelyek a jelen kiváló mérnökeinek is komoly fejtörést okoznak. Minden fejlesztés alapja először úgyszólván egy stabil és megbízható adathalmaz létrehozása.

Amennyiben a koncepcióval nem a zene felé, hanem egy iparibb környezet irányába indulnánk el, az adathalmaz átszervezésével készíthető egy ipari gépek állapotát felmérő szenzor is. Ez képes lenne megmondani a hang alapján, ha például egy gépből fogyóban van a kenőolaj, ha élettelené vált egy körfűrés (cirkula), vagy egyéb, az ipari gépek állapotával kapcsolatos előrejelzéseket adhatna.

8. IRODALOMJEGYZÉK

- [1] Valerio Velardo. *Audio Signal Processing for Machine Learning*. The Sound of AI. 2020. URL: <https://www.youtube.com/playlist?list=PL-wATfeyAMNqIee7ch3q1bh4QJFAaeNv0> (elérés dátuma 2026. 05. 23.).
- [2] Juan J. Bosch, Jordi Janer, Ferdinand Fuhrmann és Perfecto Herrera. “A Comparison of Sound Models for Automatic Instrument Recognition in Polyphonic Music Signals”. *Proceedings of the 13th International Society for Music Information Retrieval Conference (ISMIR)*. 2012, 55–60. old.
- [3] Jesse Engel, Cinjon Resnick, Adam Roberts, Sander Dieleman, Douglas Eck, Karen Simonyan és Mohammad Norouzi. “Neural Audio Synthesis of Musical Notes with WaveNet Autoencoders”. *Proceedings of the 34th International Conference on Machine Learning (ICML)*. 2017, 1068–1077. old.
- [4] Vincent Lostanlen, Joakim Andén és Mathieu Lagrange. “Extended Playing Techniques Reconstruction with Scattering Transform”. *Proceedings of the 21st International Conference on Digital Audio Effects (DAFx)*. 2018.
- [5] Carmine-Emanuele Cella, Daniele Ghisi, Philippe Esling és tsai. “TinySOL: An open dataset of orchestra instrument sounds”. *Proceedings of the International Conference on Technologies for Music Notation and Representation (TENOR)*. 2020.
- [6] Emilia Romani, Francesco Reguto, Sergio Giraldo és Rafael Ramirez. “A Real-time System for the Evaluation of Violin Sound Quality”. *Proceedings of the Sound and Music Computing Conference (SMC)*. 2015.
- [7] Karol J. Piczak. “ESC: Dataset for Environmental Sound Classification”. *Proceedings of the 23rd ACM International Conference on Multimedia*. 2015, 1015–1018. old.
- [8] Brian McFee, Colin Raffel, Dawen Liang, Daniel P Ellis, Matt McVicar, Eric Battenberg és Oriol Nieto. “librosa: Audio and music signal analysis in python”. *Proceedings of the 14th Python in Science Conference*. 2015, 18–25. old.
- [9] Martín Abadi és tsai. *TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems*. Software available from [tensorflow.org](https://www.tensorflow.org). 2015. URL: <https://www.tensorflow.org/>.

9. FÜGGELÉKEK

A függelék tartalmazza az „Automatikus hangszerfelismerés zenefelvételek alapján” című államvizsga-szakdolgozat teljes forráskódját, az adathalmaz-struktúrát és a betanított modellfájlokat tartalmazó digitális adathordozó könyvtárszerkezetének leírását, valamint a felhasznált szoftverfüggőségek összegzését.

9.1. A projekt könyvtárszerkezete

A benyújtott digitális függelék (amely a GitHubon elérhető nyilvános Git adattárban¹ is megtalálható) az alábbi főbb könyvtárakat tartalmazza. A számozott szkriptek a feldolgozási csővezeték (pipeline) egymásra épülő lépéseit jelzik.

9.1.1. táblázat: A projekt legfelső szintű könyvtárszerkezete

Könyvtár / Fájl	Leírás
thesis/	A L ^A T _E X formátumú szakdolgozat forrásfájljai, stílusfájl (sapiencia-thesis.cls), irodalomjegyzék (references.bib) és a generált main.pdf.
scripts/ owndataset/	A hangszerfelismerő rendszer Python szkriptjei (részletezés alább). A manuálisan összeválogatott és kivágott internetes hangminták hangszerosztályonként külön almappákban, valamint az újrafelvett minták a recorded_from_mic/ almappában.
hybrid_dataset_ own_final/	A végleges, összeszerelt hibrid adathalmaz 7 osztálymappával (guitar, piano, vocal, string, reed, brass, noise), osztályonként 1500 db, 1 másodperces, 16 kHz-es WAV minta.
processed_data/	Az előállított jellemzőmátrixok NumPy formátumban: X_stft_full.npy, X_log_mel_full.npy, X_mfcc_full.npy és a közös címkefájl y_labels_full.npy.
models/	A betanított Keras modellfájlok: best_stft_2dcnn_model.keras, best_log_mel_2dcnn_model.keras, best_mfcc_2dcnn_model.keras.
augmented_samples/	Az augmentációs lánc bemutató mintái: minden hangszerosztályból 3-3 eredeti-augmentált páros WAV fájl ellenőrzés céljából.
dataset/	A letöltött nyilvános referencia-adathalmazok (IRMAS, NSynth, Medley-solos-DB, TinySOL, Good-sounds, ESC-50) az elemzéshez.

¹<https://github.com/nCsab/Instrument-Recognition>

9.2. A feldolgozási csővezeték szkriptjei

A rendszer teljes feldolgozási lánc indexelt Python szkriptekből áll, amelyek a tervezés sorrendjében futtathatók. Az alábbi táblázat leírja minden szkript feladatát.

9.2.1. táblázat: A feldolgozási csővezeték szkriptjei és feladataik

Szkript	Feladat
01_analyze_datasets.py	A nyilvános referencia-adathalmazok (IRMAS, NSynth stb.) szerkezetének és statisztikáinak feltárása, összehasonlító elemzés készítése.
02_mic_prep_concatenate.py	Az újrafelvételezéshez szükséges hanganyag előkészítése: a kiválasztott minták összefűzése egyetlen hosszú WAV fájlba sávszó-szinkronizálással.
03_mic_record_audio.py	A laptop beépített mikrofonjával vagy mobiltelefon mikrofonjával történő visszajátszás és valós idejű rögzítés.
04_mic_slice_audio.py	A rögzített mikrofonos felvétel darabolása 1 másodperces szegmensekre a sávszó-szinkronizáció alapján.
05_assemble_dataset.py	A végleges hibrid adathalmaz összeállítása: a tiszta minták, az újrafelvételek és az augmentált minták egyesítése és randomizált másolása a hybrid_dataset_own_final/ könyvtárba.
05a_prepare_hybrid_generic.py	Kiegészítő előkészítő szkript a hibrid adathalmaz generikus felépítéséhez.
06_extract_features.py	A fő jellemzőkinyerő szkript, amely az összes hangmintából STFT, log-mel és MFCC spektrogramokat generál, és NumPy mátrixokba menti.
06_extract_features/	Alkönyvtár a három jellemzőtípus önálló kinyerő szkriptjeivel: 06_extract_stft.py, 06_extract_log_mel.py, 06_extract_mfcc.py.
07_realtime_recognition.py	A valós idejű hangszerfelismerő modul: mikrofonbemenet, csúszóablak, előrejelzés és megjelenítés.
10_train_model_colab/	A Google Colab felhőkörnyezetben futtatott tanító szkriptek: 10_train_stft.py, 10_train_log_mel.py, 10_train_mfcc.py.
utils/	Segédmodulok: augmentation_utils.py (az augmentációs lánc függvényei) és feature_utils.py (jellemzőkinyerési segédfüggvények).

9.3. Fejlesztői és futtatási környezet

Az alábbi táblázat tartalmazza a projekt fejlesztéséhez és futtatásához szükséges szoftverfüggőségeket.

9.3.1. táblázat: A fejlesztői környezet főbb összetevői

Szoftver / Könyvtár	Verzió	Felhasználás
Python	3.10+	A teljes feldolgozási csővezeték nyelve
TensorFlow / Keras	2.x	A 2D CNN modell építése, tanítása és mentése
Librosa	0.10+	Hangfájlok betöltése, STFT, mel-spektrogram és MFCC kinyerés
NumPy	1.24+	Jellemzőmátrixok tárolása és manipulálása
SoundFile / SoundDevice	–	Hangfájlok olvasása/írása és valós idejű mikrofon-hozzáférés
Matplotlib	3.x	Diagramok, spektrogramok és kiértékelési ábrák készítése
Google Colab	–	GPU-gyorsított modelltanítás felhőkörnyezetben
Git	2.x	Verziókezelés és a projekt előrehaladásának dokumentálása
L ^A T _E X (TeX Live 2026)	–	A szakdolgozat szedése a Sapientia sablon alapján